МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Практическое занятие № 7

Тема занятия введение в desktop-автоматизацию. создание прототипа и базовая автоматизация формы.
Вариант: 11

Выполнил студент 3 курса группы
ЭФБО-09-23 Калинин Н.В.

Проверил Громов Е.В.

Москва
2026 г.

Цель работы: сформировать у студентов практические навыки создания desktop-приложений с заданным интерфейсом и разработки базовых сценариев их автоматизации с использованием платформы Puzzle RPA, а также освоение фундаментальных принципов промышленной роботизации: синхронизация с интерфейсом, проверка состояний элементов, логирование и обработка типовых ошибок.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Основные понятия

Автоматизация деятельности человека с использованием программных роботов является одним из наиболее динамично развивающихся направлений цифровой трансформации бизнеса. Особое место в RPA занимает автоматизация взаимодействия с настольными приложениями (Desktop Automation), поскольку значительная часть корпоративного программного обеспечения представлена именно desktop-решениями, многие из которых не имеют программных интерфейсов (API) для интеграции.

В центре нашего внимания – автоматизация desktop-приложений. Почему именно desktop?

Несмотря на активное развитие веб-технологий, значительная часть корпоративного программного обеспечения по-прежнему представлена настольными приложениями. Бухгалтерские системы, CRM, документооборот, специализированное ПО для складов и производства – всё это зачастую работает на компьютерах сотрудников как отдельные программы. У многих из них нет современных API, веб-интерфейсов или возможностей интеграции. Но у них есть графический интерфейс, с которым работает человек. И значит, с ним может работать и робот.

В рамках данной практической работы студентам предстоит освоить базовые принципы и инструменты Desktop-автоматизации на платформе Puzzle RPA, создав прототип приложения и реализовав сценарий его автоматизации.

К концу этой практической работы вы будете:

Знать:

- Как робот "видит" desktop-приложение и взаимодействует с ним.
- Какие блоки Desktop-автоматизации существуют в Puzzle RPA и для чего каждый из них предназначен.
- Как правильно настраивать селекторы элементов через Мастер UI.
- Какие типовые ошибки возникают при автоматизации и как их избегать.
- Чем Desktop-автоматизация отличается от Web-автоматизации.

Уметь:

- Запускать desktop-приложение из RPA-сценария.
- Ожидать загрузки интерфейса и появления элементов.
- Заполнять формы, выбирать значения из списков, нажимать кнопки.
- Читать состояние элементов и принимать решения на основе полученных данных.
- Извлекать данные из таблиц для верификации результатов.
- Вести журнал выполнения и сохранять его в файл.

Владеть:

- Навыками работы с Мастером UI.
- Методикой построения надежных селекторов.
- Приемами отладки RPA-сценариев.
- Культурой логирования и документирования процессов автоматизации.

Общая концепция Desktop-автоматизации

Вы уже изучили основы RPA: познакомились с платформой Puzzle RPA, узнали, что такое переменные, циклы, условия, научились создавать простые сценарии. И сейчас, вероятно, задаётесь вопросом: "А почему мы начинаем погружение в автоматизацию именно с Desktop, а не с более привычного и, казалось бы, простого Web?"

Это закономерный вопрос. В современном мире мы окружены веб-интерфейсами, и, кажется, что автоматизировать сайты проще и нужнее. Но давайте посмотрим на ситуацию шире.

Почему Desktop – это фундамент, а не исключение

Аргумент 1. Исторический приоритет

Большинство корпоративных систем были созданы в эпоху, когда веб-технологии только зарождались. Бухгалтерские системы 1990-х, складские учёты 2000-х, CRM-системы начала 2010-х – всё это создавалось как desktop-приложения. Они работают до сих пор и менять их никто не собирается.

Аргумент 2. Сложность порождает мастерство

Desktop-автоматизация объективно сложнее веб-автоматизации. В вебе есть чёткие стандарты (HTML, CSS, DOM), встроенные инструменты разработчика, предсказуемая структура страниц. Desktop – это "дикий запад", где каждое приложение живёт по своим законам. Освоив Desktop, вы легко перейдёте к Web. Но не наоборот.

Аргумент 3. Мы идём от сложного к простому

Наш учебный курс построен по принципу "сначала сложное, потом простое". Вы пройдёте через все трудности Desktop-автоматизации, научитесь решать нетривиальные проблемы, и когда мы перейдём к Web (на следующих занятиях), он покажется вам лёгкой прогулкой.

Аргумент 4. Desktop – это проверка на профпригодность

Умение автоматизировать сложное desktop-приложение – это маркер профессионального RPA-разработчика. Заказчики ценят специалистов, которые могут "подружиться" с любой старой системой, а не только с современными сайтами.

Desktop-автоматизация – это технология создания программных роботов, которые взаимодействуют с графическим пользовательским интерфейсом (GUI) настольных приложений, имитируя действия человека-оператора: запуск программ, ввод данных, нажатие кнопок, чтение информации с экрана, копирование данных между приложениями.

Возьмём типичную задачу: "Перенести данные из Excel в программу учёта клиентов".

Как это делает человек:

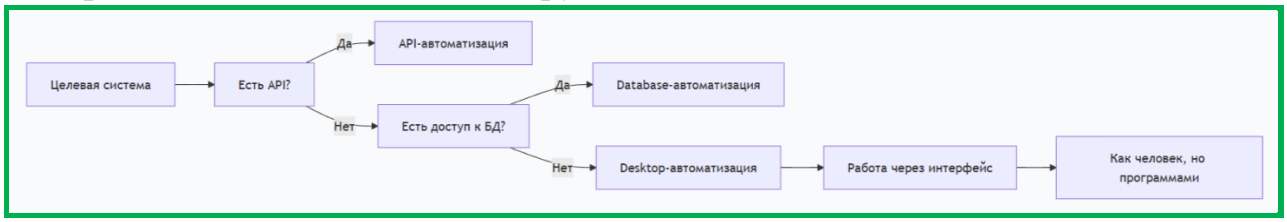
- Открывает Excel, копирует имя клиента.
- Переключается в программу учёта.
- Нажимает кнопку "Добавить клиента".
- Вставляет имя в поле "Наименование".
- Заполняет остальные поля.
- Нажимает "Сохранить".
- Как это делает робот (Desktop-автоматизация):
- Блоком Открыть приложение запускает программу учёта.
- Блоком Ожидать появления элемента ждёт загрузки окна.
- Блоком Клик по элементу нажимает кнопку "Добавить клиента".
- Блоком Ввод в приложение заполняет поля данными из Excel.
- Блоком Клик по элементу нажимает "Сохранить".

Ключевой момент: Робот не имеет доступа к внутренней логике программы учёта. Он не знает, как устроена её база данных, какие там таблицы и поля. Он просто "смотрит на экран" и делает то же, что и человек.

Когда Desktop-автоматизация незаменима:

Ситуация	Пример	Почему не подходят другие методы
У программы нет API	Старая бухгалтерская система 1990-х	API просто не существует
API есть, но доступ ограничен	SAP, 1С – лицензии на API дороги	Дешевле посадить робота
Программа работает только с GUI	Графические редакторы, CAD-системы	Интерфейс – единственный способ
Сценарий требует "глаз"	Проверка, появилось ли сообщение	Нужно видеть то, что видит пользователь

Принципиальное отличие от других видов автоматизации:



Desktop-автоматизация – это всегда запасной вариант, который срабатывает, когда все остальные методы недоступны. Но именно этот "запасной вариант" покрывает 80% реальных бизнес-задач.

Как робот воспринимает интерфейс?

Давайте проведём эксперимент.

Инструкция: Закройте глаза. Не подглядывайте. А теперь ответьте на вопросы:

Какого цвета кнопка "Пуск" в Windows?

Есть ли на ней иконка?

Она круглая или квадратная?

Скорее всего, вы не сможете ответить точно. Но если я попрошу вас нажать кнопку "Пуск" с закрытыми глазами – вы, вероятно, справитесь. Вы знаете, где она примерно находится, знаете, что на неё нужно нажать.

Робот находится ровно в такой же ситуации. Он не видит цвета, иконки, тени, градиенты. Но он может:

- Найти элемент по его координатам (ненадёжно).
- Найти элемент по его свойствам (надёжно).
- "Потрогать" элемент – прочитать его свойства.
- Воздействовать на элемент – кликнуть, ввести текст.

Для робота UI-элемент – это не "красивая кнопка с надписью", а структура данных, содержащая:

```json
{
  "AutomationId": "btnAddRequest",
  "Name": "Add Request",
  "ClassName": "Button",
  "ControlType": "Button",
  "IsEnabled": true,
  "IsVisible": true,
  "IsFocused": false,
  "BoundingRectangle": {
    "X": 100,
    "Y": 200,
    "Width": 120,
    "Height": 30
  },
  "ProcessId": 12345,
  "RuntimeId": "42.123456.7890"
}
```

Категории свойств

Категория 1. Идентификационные свойства

Позволяют отличить один элемент от другого.

Свойство	Описание	Пример	Надёжность
`AutomationId`	Уникальный ID, заданный разработчиком	`txtFirstName`	□□□□□
`Name`	Текст, который видит пользователь	`"Имя пользователя"`	□□□□
`ClassName`	Тип элемента	`"TextBox", "Button"`	□□
`ControlType`	Роль элемента	`"Edit", "Button"`	□□

Важно: `AutomationId` – это "паспорт" элемента. Если разработчик его задал – вам повезло. Если нет – придётся комбинировать другие свойства.

Категория 2. Пространственные свойства

Где элемент находится на экране.

Свойство	Описание	Пример
`BoundingRectangle`	Координаты и размер	`X=100, Y=200, Width=120, Height=30`
`Location`	Положение	`{X=100, Y=200}`
`Size`	Размер	`{Width=120, Height=30}`

Важно: Никогда не ищите элементы по координатам! Окно можно передвинуть, разрешение экрана может измениться. Координаты используем только для вспомогательных действий (прокрутка, смещения).

Категория 3. Свойства состояния

Что с элементом сейчас происходит.

Свойство	Описание	Что означает для робота
`IsEnabled`	Доступен для взаимодействия	Можно кликать / нельзя

Свойство	Описание	Что означает для робота
`IsVisible`	Видим на экране	Есть смысл искать / нет
`IsFocused`	В фокусе ввода	Сюда будет вводиться текст
`IsOffscreen`	Скрыт за пределами экрана	Нужно прокрутить

Категория 4. Содержательные свойства

Данные, которые содержит элемент.

Свойство	Описание	Для каких элементов
`Text`	Текст элемента	Надписи, поля ввода
`Value`	Значение	Прогресс-бары, ползунки
`SelectedItem`	Выбранный пункт	Списки, комбобоксы
`IsChecked`	Отмечен ли	Чекбоксы, радиокнопки

Фундаментальные принципы Desktop-автоматизации

Принцип иерархичности

Формулировка: Любой UI-элемент существует в иерархии. Окно содержит панели, панели содержат группы, группы содержат кнопки.

Следствие: Искать элемент можно двумя способами:

- Прямой поиск – по уникальному идентификатору (быстро, но требует уникальности).
- Иерархический поиск – найти окно, в нём панель, в панели кнопку (медленнее, но надёжнее при неуникальных ID).

Пример: python

- Прямой поиск (если `AutomationId` уникален)

```
кнопка = Найти_элемент("btnSave")
```

- Иерархический поиск (если ID неуникален)

```
окно = Найти_окно("MainForm")
панель = Найти_внутри(окно, "Panel_Buttons")
кнопка = Найти_внутри(панель, "Button", "Сохранить")
```

Принцип уникальности идентификации

Формулировка: Селектор элемента должен однозначно идентифицировать один и только один элемент в контексте поиска.

Проблема: В приложении может быть 10 кнопок с `ClassName = "Button"`. Какая из них нужна?

Решения:

- Использовать `AutomationId` (он должен быть уникальным по замыслу).
- Комбинировать свойства: `ClassName` + `Name` + позиция.
- Использовать иерархический поиск (уточнять контекст).

Принцип независимости от представления

Формулировка: Селектор не должен зависеть от того, как элемент выглядит визуально.

Плохой селектор: "Кнопка синего цвета слева вверху".

Хороший селектор: "Элемент с `AutomationId = 'btnAdd'`".

Почему: Визуальное представление может меняться (тема оформления, разрешение экрана), а идентификаторы остаются стабильными.

Принцип минимальной достаточности

Формулировка: Селектор должен содержать ровно столько свойств, сколько необходимо для уникальной идентификации, и ни одного лишнего.

Пример:

- Если `AutomationId` уникален: достаточно его одного.
- Если `AutomationId` неуникален, но `Name` уникален в пределах контейнера: `AutomationId + Name`.
- Если `Name` тоже неуникален: добавляем `ClassName`, позицию в иерархии.

Почему это важно: Чем сложнее селектор, тем он более хрупкий. Любое изменение в приложении может его сломать.

Принцип отказоустойчивости

Формулировка: Робот должен быть готов к тому, что что-то пойдёт не так.

Уровни отказоустойчивости:

- Обнаружение – робот понимает, что элемент не найден (таймауты).
- Логирование – робот записывает факт ошибки.
- Повтор – робот пытается выполнить действие ещё раз.
- Альтернатива – робот пробует другой способ.
- Эскалация – робот останавливается и зовёт человека.

Desktop в корпоративном ландшафте: где это применяется

Отрасль	Сценарий	Приложение
Банки	Выгрузка выписок	АБС (Автоматизированные банковские системы)
Бухгалтерия	Проведение документов	1С, SAP, Парус
Логистика	Оформление накладных	Складские системы
HR	Заведение сотрудников	Кадровые системы
IT	Регистрация заявок	Service Desk

Почему компании выбирают Desktop-автоматизацию?

1. Нельзя менять систему – старая, но стабильная, любое изменение рискованно.

2. Нет бюджета на модернизацию – дешевле посадить робота.

3. Вендор не даёт API – закрытая система.

4. Скорость внедрения – робота можно создать за недели, а доработку системы ждать месяцы.

Реальный пример из практики.

Кейс: Крупный ретейлер, 500 магазинов. Каждый вечер менеджеры выгружают отчёты из кассовой программы (desktop-приложение 2005 года) и загружают в центральную систему.

Проблема: Ручной ввод занимает 2 часа в день на магазин. 1000 человеко-часов ежедневно.

Решение: RPA-робот, который:

- Запускает кассовую программу.
- Формирует отчёт.
- Сохраняет в Excel.
- Отправляет по почте.
- Закрывает программу.

Результат: Экономия 1000 часов в день. Окупаемость – 2 месяца.

Семь главных идей, которые нужно запомнить:

- Desktop-автоматизация – это не "взлом", а имитация действий человека через интерфейс.
- Робот "слепой" – он не видит картинку, а "ощупывает" элементы через их свойства.
- Свойства элемента – это единственный способ для робота понять, что перед ним.
- Селектор – это "фоторобот" элемента, составленный из свойств.
- Синхронизация важнее скорости – лучше подождать лишнюю секунду, чем упасть с ошибкой.
- Каждое приложение уникально – нет единого рецепта, есть только принципы.
- Надёжность достигается через – правильные селекторы, синхронизацию и обработку исключений.

Критические отличия Desktop-автоматизации от Web-автоматизации

Представьте, что вы учитесь водить машину. Вы можете научиться водить легковой автомобиль, а можете – грузовик. Принципы похожи: руль, педали, коробка передач. Но управление грузовиком требует учёта множества дополнительных факторов: габариты, нагрузка на ось, тормозной путь с прицепом.

Примерно так же соотносятся Web- и Desktop-автоматизация.

На первый взгляд, и там, и там нужно:

- Находить элементы на экране.
- Кликать по ним.
- Вводить текст.
- Читать данные.

Но глубина, сложность и "подводные камни" в Desktop-автоматизации на порядок серьёзнее. Освоив Desktop, вы легко перейдёте к Web. Но если начать с Web, переход к Desktop может стать шоком.

Веб-технологии жёстко стандартизированы. Существует консорциум W3C, который разрабатывает и утверждает стандарты HTML, CSS, DOM. Браузеры (Chrome, Firefox, Edge) стараются эти стандарты соблюдать.

Что это даёт автоматизации:

Аспект	Почему это хорошо
Единая модель DOM	Все веб-страницы строятся по одним правилам
Предсказуемая структура	Теги, атрибуты, вложенность – всё подчиняется логике
Стандартные селекторы	XPath, CSS-селекторы работают везде одинаково
Инструменты разработчика	Встроены в браузер, мощные и удобные

Пример веб-элемента:

```
html
<input type="text" id="username" name="username" class="form-control" placeholder="Введите имя">
```

У этого элемента есть:

`id` – гарантированно уникальный в пределах страницы.

`name` – часто уникальный.

`class` – для стилей, но тоже можно использовать.

`type` – определяет поведение.

`placeholder` – подсказка.

Для автоматизации это рай. У вас есть множество атрибутов, любой из которых может стать надёжным селектором.

В мире Desktop нет единого стандарта. Приложения пишутся на разных фреймворках, разными разработчиками, в разное время.

С чем приходится сталкиваться:

Фреймворк	Особенности автоматизации
WinForms	AutomationId может быть осмысленным, а может отсутствовать
WPF	Сложная иерархия, много вложенных элементов
Qt	Своя модель доступности, не всегда совместимая с UIA
Java Swing	Требует Java Access Bridge, AutomationId генерируется автоматически
MFC (старые)	Минимальная поддержка автоматизации
Самописные	Могут вообще не предоставлять информацию об элементах

Пример desktop-элемента в инспекторе:

```
xml
<Button AutomationId="128473" Name="Сохранить" ClassName="Button" ControlType="Button"/>
```

Проблемы видны сразу:

`AutomationId "128473"` – сгенерирован автоматически, может измениться при следующей компиляции.

`Name "Сохранить"` – может быть неуникальным (в приложении 10 кнопок "Сохранить").

`ClassName "Button"` – одинаков у всех кнопок.

Главный вывод. В Web вы ищете элемент по его паспорту. В Desktop вы часто собираете фоторобот по кусочкам.

Ниже представлена сравнительная таблица возможностей поиска элементов в Web и Desktop. Когда вы будете её изучать, обратите внимание не просто на "галочки", а на глубину различий:

- В Web у вас всегда есть выбор. Если не работает один способ поиска, вы используете другой. В Desktop выбора часто нет – работаете с тем, что даёт приложение.
- В Web есть частичный поиск, регулярные выражения, навигация по соседству – всё это делает автоматизацию гибкой и устойчивой к изменениям. В Desktop таких инструментов нет.
- В Web вы ищете элемент. В Desktop вы часто выслеживаете его, как охотник, собирая по крупицам доступную информацию.

Возможность	Web	Desktop
Поиск по уникальному ID	✅ Всегда	Не всегда
Поиск по тексту	✅ Полный текст	✅ Только Name
Поиск по части текста	✅ contains()	❌ Нет
Поиск по атрибутам	✅ Любые	❌ Только стандартные
Поиск по позиции	✅ index()	✅ Через иерархию
Регулярные выражения	✅	❌
Относительный поиск	✅ Оси XPath	✅ Только "внутри"
Поиск по соседству	✅ following-sibling	❌

В Web вы выбираете самый удобный селектор из десятка возможных. В Desktop вы используете то, что есть, и надеетесь, чтобы это не изменилось в следующей версии.

Устойчивость к изменениям окружения

Web: Независимость от экрана

Веб-страницы адаптивны. Они могут:

- Менять размер под окно браузера.
- Перестраивать элементы для мобильных устройств.
- Масштабироваться без потери функциональности.

Для автоматизации это означает:

- Элементы всегда там, где ожидаются по DOM.
- Разрешение экрана не влияет на поиск.
- Можно масштабировать страницу – элементы всё равно найдутся.

Desktop: Чувствительность к окружению

Desktop-приложения жёстко привязаны к окружению:

Фактор	Влияние
Разрешение экрана	Элементы могут "разъезжаться"
DPI-масштабирование	Размеры и положения меняются
Тема оформления	Цвета, отступы, размеры
Положение окна	Координаты всех элементов меняются
Версия ОС	Может меняться отрисовка

Координатный подход: смертный грех в Desktop.

В Web клики по координатам почти не используются – это моветон.

В Desktop новички часто пытаются кликать по координатам, потому что "элемент не находится".

Чем это опасно: плохой код (Desktop) –- клик_по_координатам(100, 200) привязано к пикселям

- Если окно сдвинуто – клик мимо.
- Если разрешение другое – клик мимо.
- Если DPI масштабирование 150% – клик мимо.
- Если приложение обновили и кнопка сместилась – клик мимо.

Отладка и диагностика

Web: Инструменты разработчика

В любом браузере есть мощнейшие инструменты разработчика (F12):

Возможность	Что даёт
Инспектор элементов	Показать DOM элемента
Консоль	Выполнять JavaScript, видеть ошибки
Сеть	Видеть все запросы
Источники	Отладка JavaScript
Свойства	Все атрибуты элемента

Desktop: Мастер UI

В Desktop у нас есть только Мастер UI:

Возможность	Что даёт
Дерево элементов	Иерархия, но не всегда понятная
Свойства	Ограниченный набор
Подсветка	Показать, где элемент на экране
Тестирование	Проверить, найдётся ли элемент

Чего НЕТ в Desktop:

- Нет консоли для выполнения кода.
- Нет просмотра сетевых запросов.
- Нет отладки приложения.
- Нет информации о внутренних процессах.

В Web вы можете быстро понять, почему элемент не находится – открыли консоль, посмотрели DOM, увидели, что ID изменился.

В Desktop вы часто действуете вслепую. Элемент может не находиться по тысяче причин, и единственный способ диагностики – перепроверять селектор и гадать.

Сравнительная таблица: Web vs Desktop (полная версия)

Критерий	Web-автоматизация	Desktop-автоматизация
Стандартизация	Единые стандарты W3C	Множество фреймворков
Идентификация	ID, Name, XPath, CSS, текст	AutomationId, Name, ClassName
Уникальность элементов	ID гарантированно уникален	AutomationId может отсутствовать
Поиск по тексту	Полнотекстовый, contains()	Только точное совпадение Name
События готовности	load, DOMContentLoaded	Нет, только опрос состояния
Модальные окна	Блокируют вкладку	Блокируют всё приложение
Потеря фокуса	Редко	Часто
Зависимость от экрана	Нет	Высокая
Инструменты отладки	F12 (мощные)	Инспектор UI (ограничен)
Скорость выполнения	Высокая	Средняя (нужны ожидания)
Сложность разработки	Средняя	Высокая
Надёжность	Высокая	Средняя (зависит от приложения)

Семь главных отличий, которые нужно запомнить:

- Стандартизация – Web стандартизирован, Desktop фрагментирован.
- Идентификация – В Web богатый выбор селекторов, в Desktop ограниченный набор свойств.
- Синхронизация – В Web есть события загрузки, в Desktop только опрос состояния.
- Модальность – В Web блокируется вкладка, в Desktop блокируется всё приложение.
- Фокус – В Web стабилен, в Desktop может быть потерян в любой момент.
- Окружение – Web не зависит от экрана, Desktop критически зависит.
- Отладка – В Web мощные инструменты, в Desktop только Инспектор.

Теперь вы понимаете, почему наш курс построен именно так. Мы начали с Desktop не потому, что он проще, а потому что он сложнее. Освоив Desktop, вы:

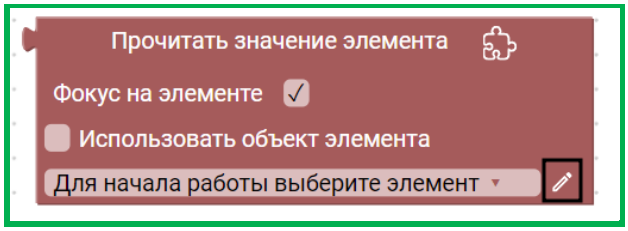
- Научитесь тщательно продумывать селекторы.
- Привыкнете к обязательной синхронизации.
- Натренируете "чувство опасности" в отношении фокуса и модальности.
- Поймёте, как важно логировать каждое действие.

И когда мы перейдём к Web, он покажется вам лёгкой прогулкой.

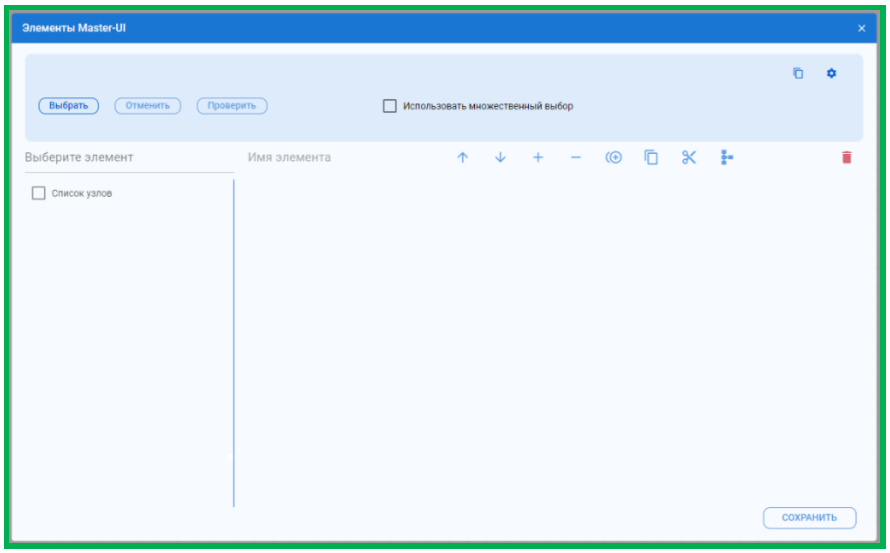
Работа с Мастером UI: продвинутые техники

Мастер UI для работы с десктопными приложениями – встроенный инструмент Puzzle RPA, который позволяет извлекать свойства пользовательского интерфейса (UI)–элементов десктопных приложений.

Чтобы открыть Мастер UI для десктопных приложений, требуется кликнуть по кнопке, расположенной на блоках из раздела Десктопные приложения:



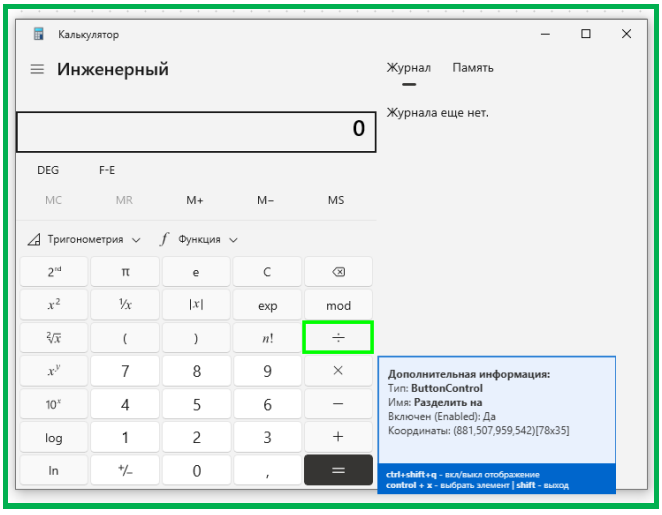
Стартовый экран:



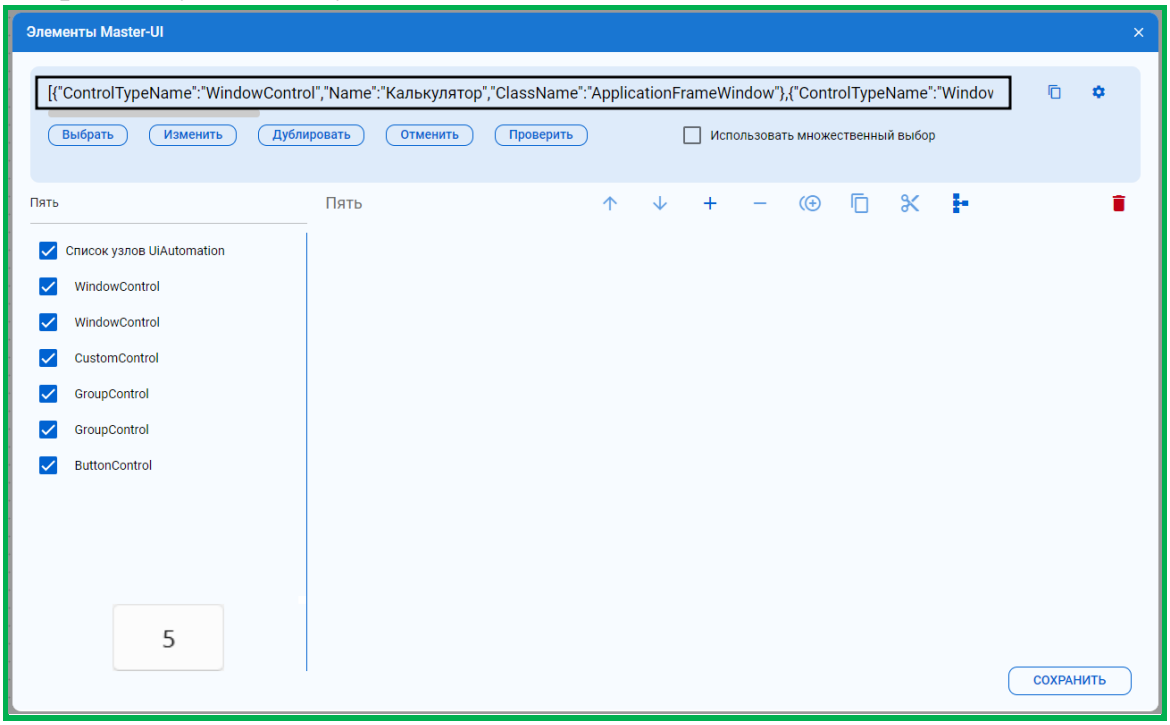
Нажатие на кнопку «Выбрать» запускает процесс выбора элемента. Для выбора нескольких элементов необходимо предварительно поставить отметку в чекбокс: Использовать множественный выбор.

После нажатия на кнопку Выбрать требуется:

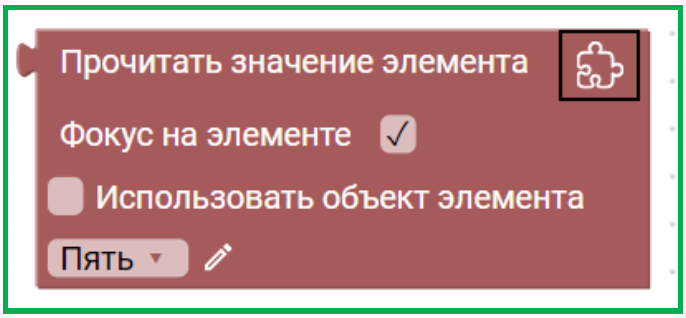
- Навести курсор мыши на интересующий элемент;
- Нажать комбинацию *Ctrl+X* для выбора элемента (или другую установленную в настройках Мастер UI);
- После выделения элемента зеленой рамкой, для подтверждения нажать *Shift*.



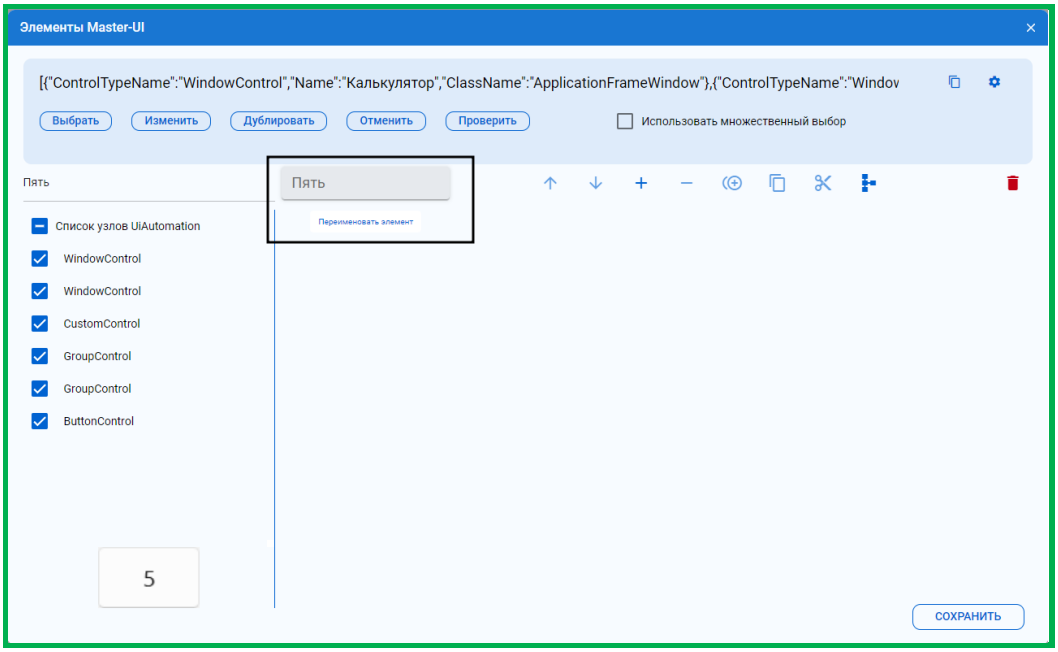
- После выполнения данных действий, в окне редактора отобразится Путь к выбранному элементу:



Запустить процесс выбора элемента можно с помощью кнопки на блоке:



Выбранному элементу по умолчанию дается название `element_n` или значение атрибутов `title` или `Name`. Переименовать элемент можно, введя новое название в выделенное поле:

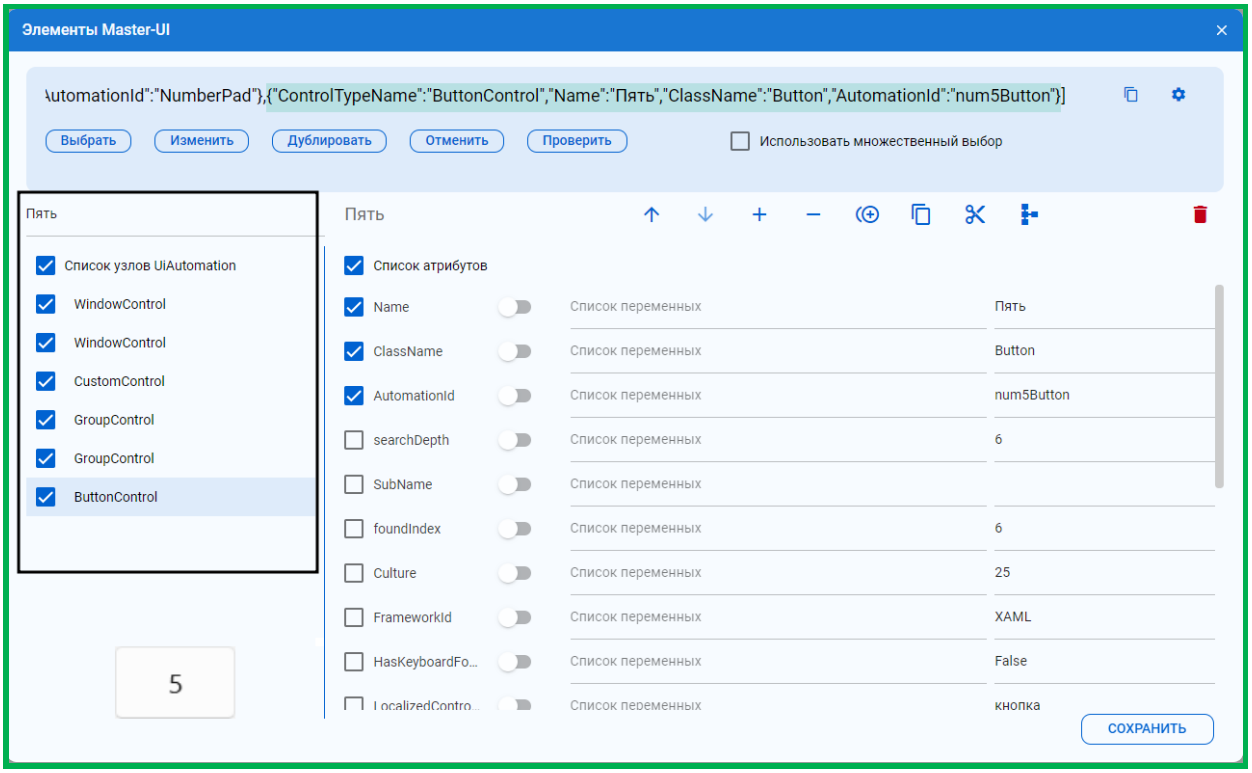


Редактор также имеет следующие кнопки:

- Изменить – Позволяет перевыбрать элемент;
- Дублировать – Позволяет создать дубликат выбранного элемента;
- Отменить – Позволяет отменить изменения узлов и атрибутов.
- Проверить – Позволяет проверить, какой элемент выбран. При нажатии элемент будет обведен зеленой рамкой, а в поле для отображения пути к элементу отобразится количество найденных элементов.

В левой части редактора располагается «Список узлов». С помощью чекбоксов можно удалять и добавлять узлы из пути к элементу. Не рекомендуется снимать отметку из чекбокса WindowControl.

Под списком узлов располагается скриншот выбранного элемента.

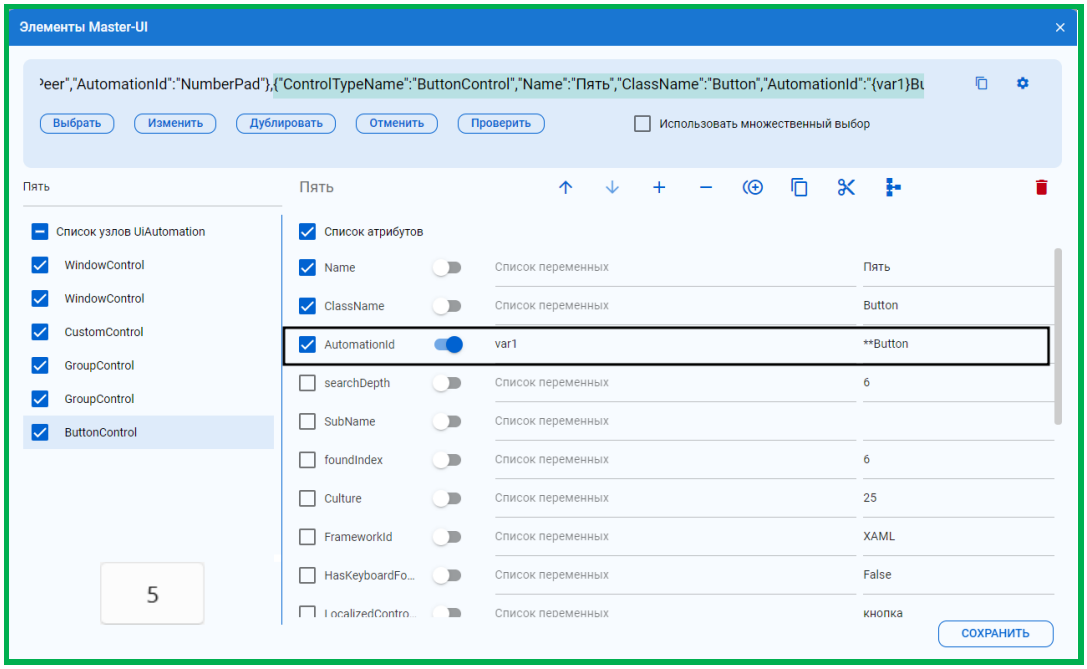


Кликом можно выбрать определенный узел для его настройки. При этом Список атрибутов и их значения выбранного узла отобразятся в правой части редактора. С помощью чекбоксов можно удалять атрибуты из узла и добавлять их обратно.

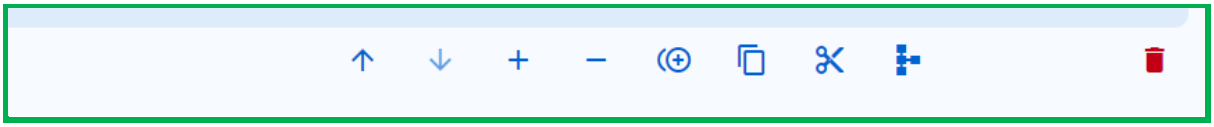
Для использования переменной в качестве значения атрибута, необходимо:

- Активировать переключатель рядом с названием выбранного атрибута;
- Выбрать переменную из списка справа от переключателя.

Если переменную необходимо использовать как часть значения атрибута, необходимо указать её расположение с помощью символов ** в поле Значение атрибута.



Над списком атрибутов расположены кнопки работы с узлами и атрибутами.



Они позволяют:

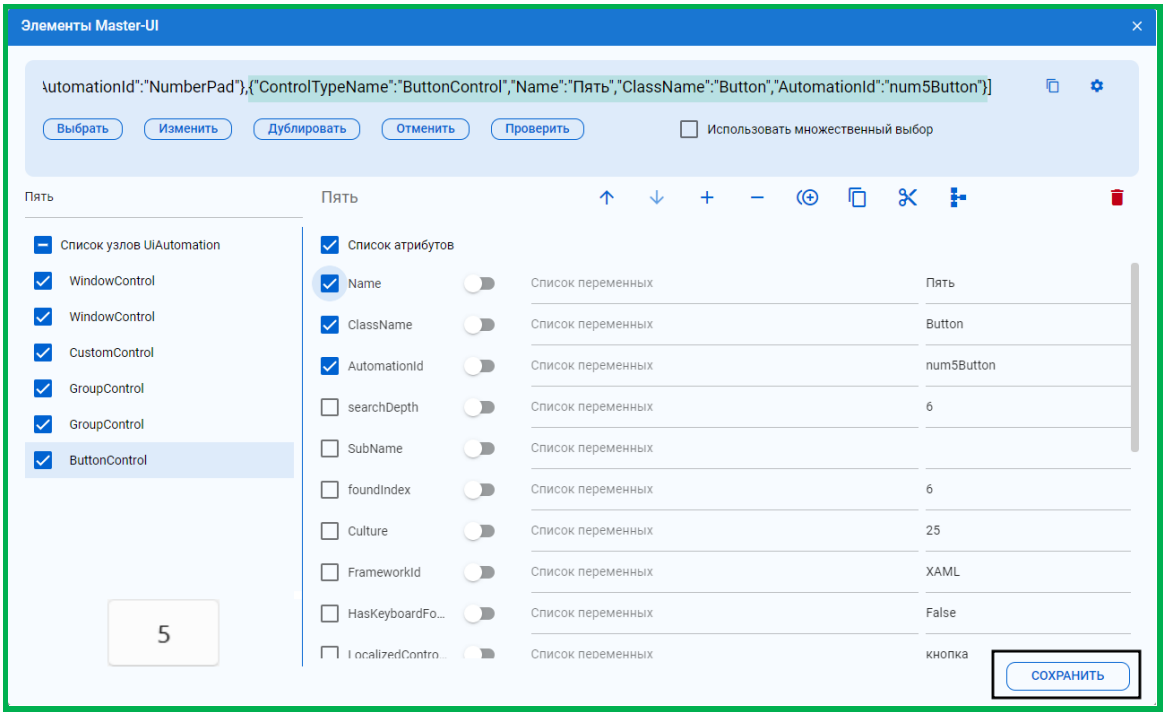
- Переместить узел вверх;
- Переместить узел вниз;
- Добавить узел;
- Удалить узел;
- Дублировать узел;
- Скопировать узел;
- Вырезать узел;
- Открыть дерево элементов (Puzzle Inspect)–Продвинутый инструмент для работы с деревом элементов десктопных приложений;
- Удалить элемент – Позволяет удалить один выбранный или все неиспользуемые (в алгоритме программного робота) элементы.

При выборе функции “Удалить все неиспользуемые элементы”, будут удалены все неиспользуемые элементы во всем проекте, а не только в выбранном процессе.

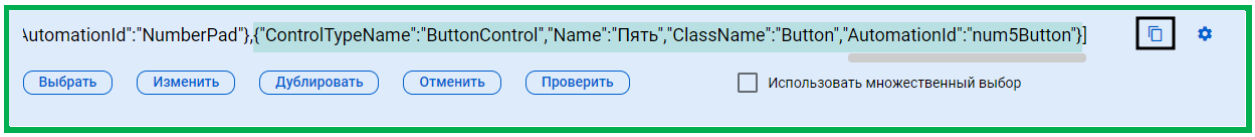
Для удобства предусмотрена клавиатурная навигация по интерфейсу:

- ↑ (стрелка вверх) / ↓ (стрелка вниз)–перемещение между узлами выбранного элемента;
- Enter – включение / отключение выбранного узла.

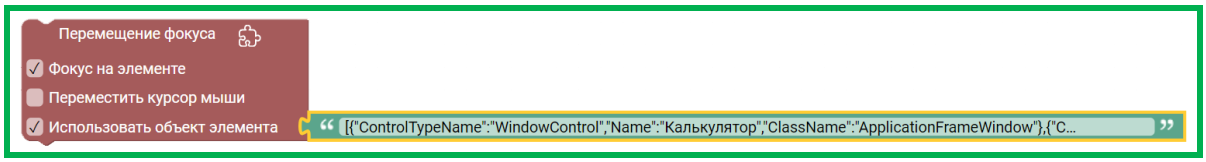
Для сохранения изменений необходимо выполнить клик по кнопке Сохранить в нижней правой части редактора.



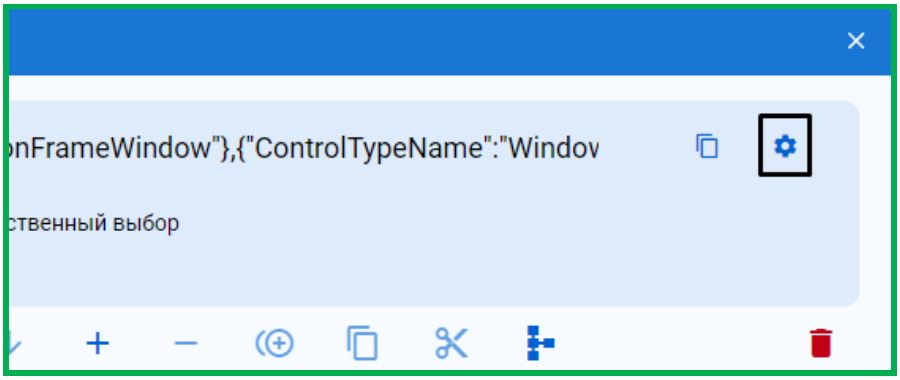
Если требуется Скопировать полученный путь, то необходимо нажать на соответствующую кнопку в верхней правой части редактора.



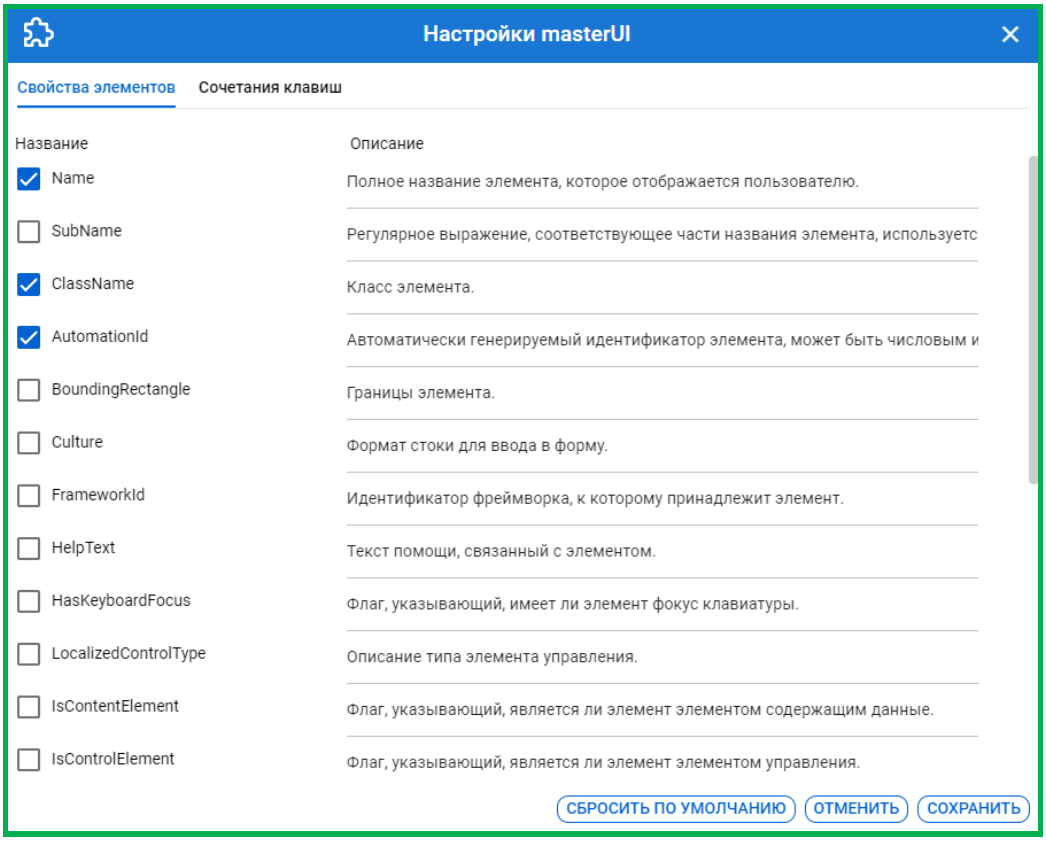
Скопированную JSON-строку можно использовать в блоках из категории Десктопные приложения с активированной функцией Использовать объект элемента.



Нажатие на символ Настройки открывает меню настроек Мастер UI.

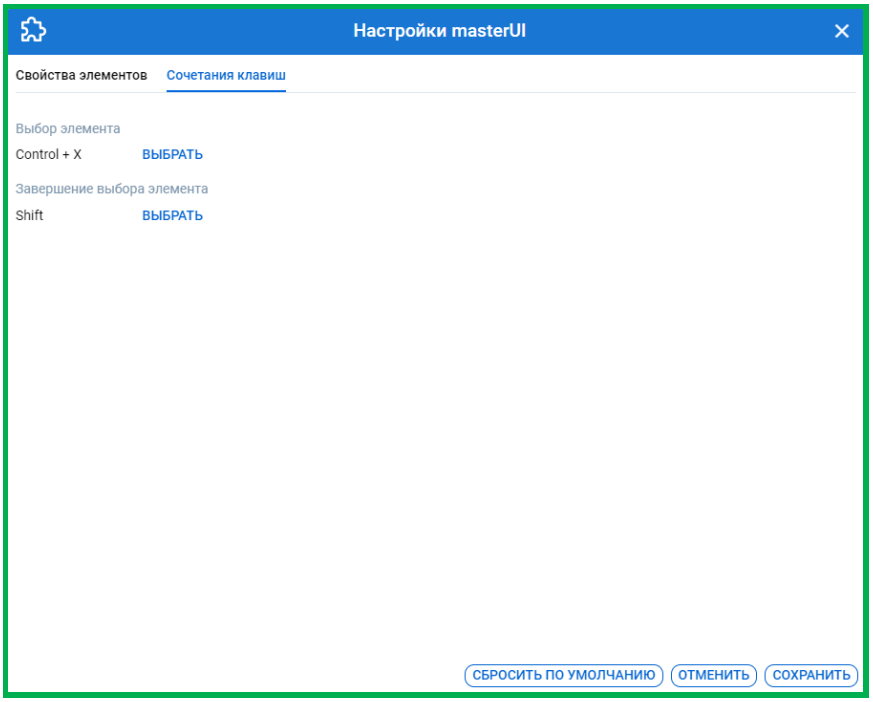


На вкладке Свойства элементов располагается список, позволяющий выбрать свойства по умолчанию при помощи чекбоксов. Описания свойств располагаются рядом.



На вкладке Сочетания клавиш располагается список используемых горячих клавиш для выбора UI элементов.

Нажатие кнопки Выбрать открывает режим изменения сочетания клавиш.

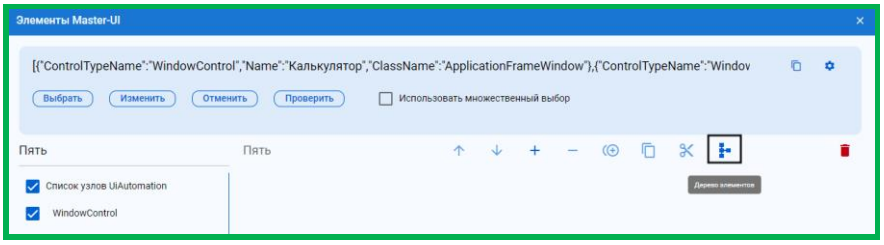


В платформе Puzzle RPA также существует инструмент под названием "Puzzle инспектор" (Puzzle Inspect), который предназначен специально для работы с десктопными приложениями.

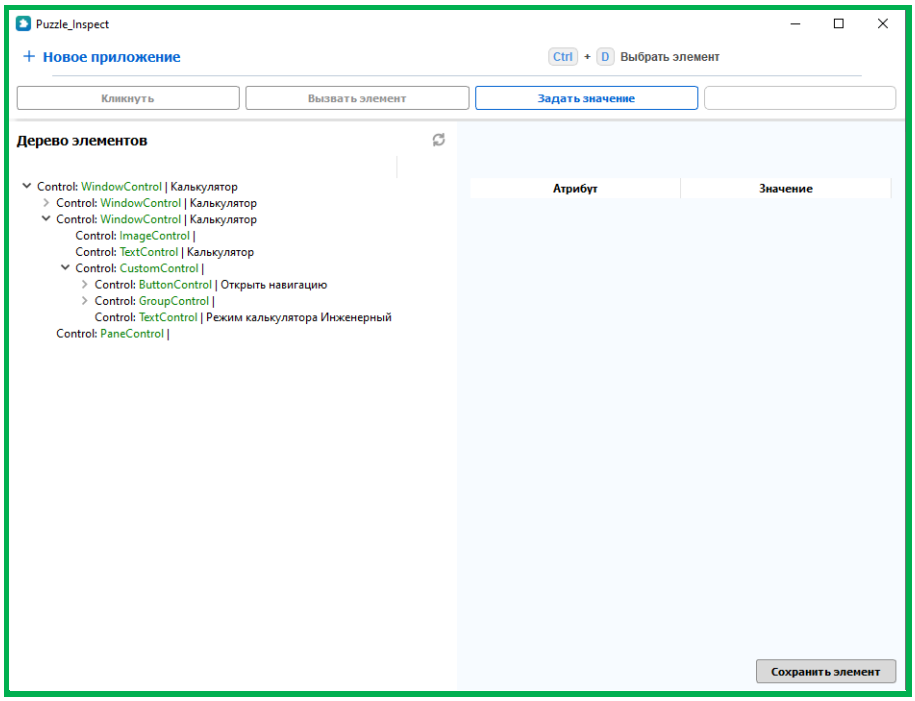
Puzzle инспектор для десктопных приложений – встроенный инструмент, который позволяет просматривать полную иерархию элементов (UI-дерева) выбранного десктопного приложения. Структура интерфейса представляется в виде дерева, где каждый узел соответствует конкретному элементу окна – такому как

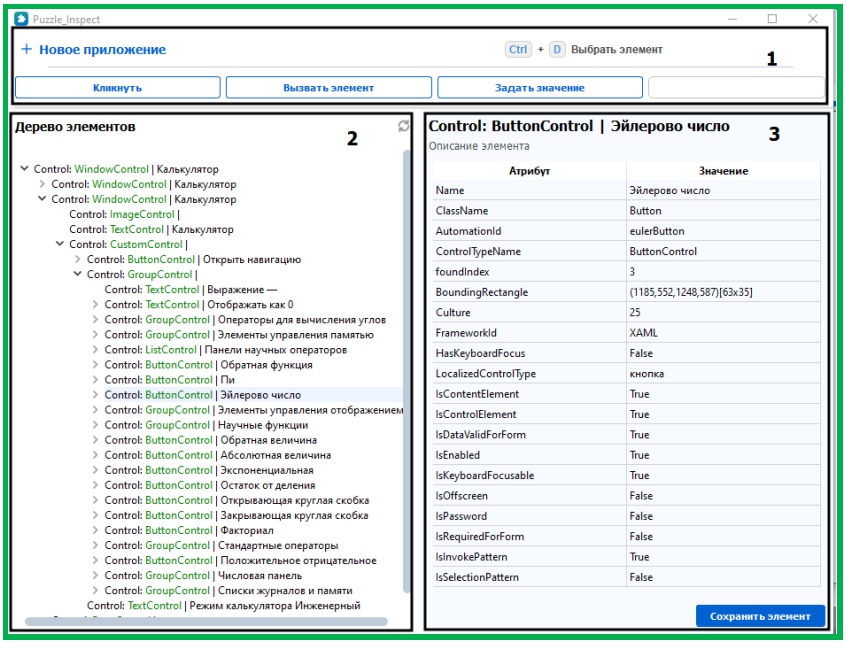
кнопка, текстовое поле, таблица и т.д. Для каждого узла доступны его атрибуты и соответствующие значения, что позволяет анализировать свойства элементов интерфейса.

Чтобы открыть *Puzzle инспектор для десктопных приложений*, требуется кликнуть по кнопке, расположенной в интерфейсе Мастер UI для работы с десктопными приложениями:



Стартовый экран





1 – Панель взаимодействия с приложением

В верхней части расположены кнопки для взаимодействия с десктопнымы приложениями:

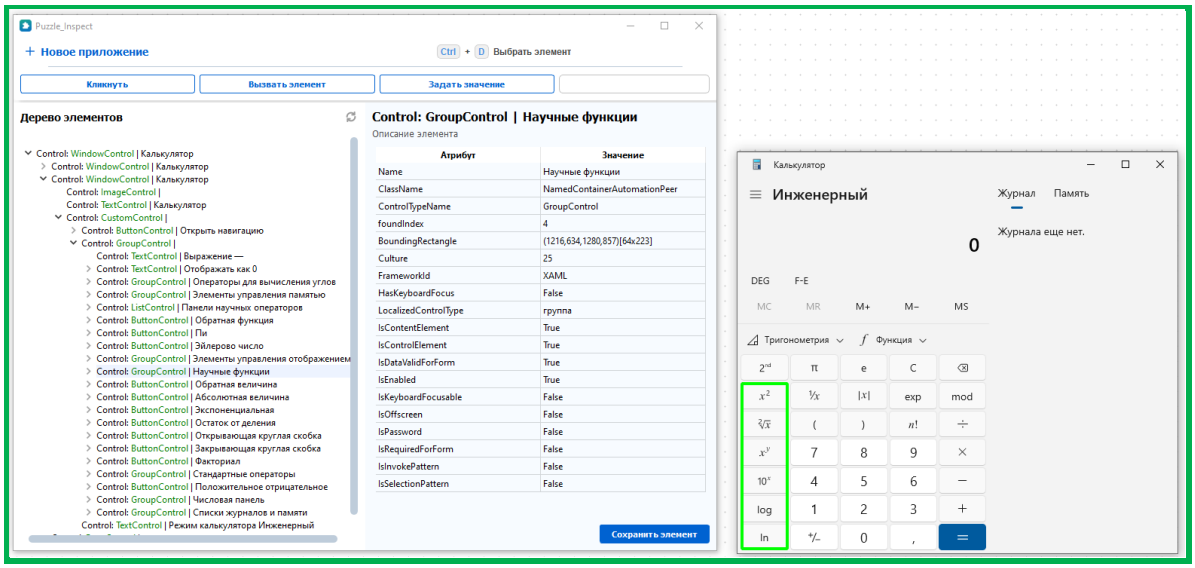
- + Новое приложение – позволяет выбрать новое приложение;
- Кликнуть – выполняет клик по выбранному элементу;
- Вызвать элемент – выполняет вызов выбранного элемента;
- Задать значение – устанавливает текстовое значение для выбранного элемента.

2 – Дерево элементов

В левой части расположено дерево элементов, позволяющее просматривать полную иерархию элементов выбранного десктопного приложения.

Кнопка Обновить (или комбинация клавиш CTRL + R) выполняет обновление дерева элементов.

При клике на узел, происходит визуальное выделение соответствующего элемента в приложении.

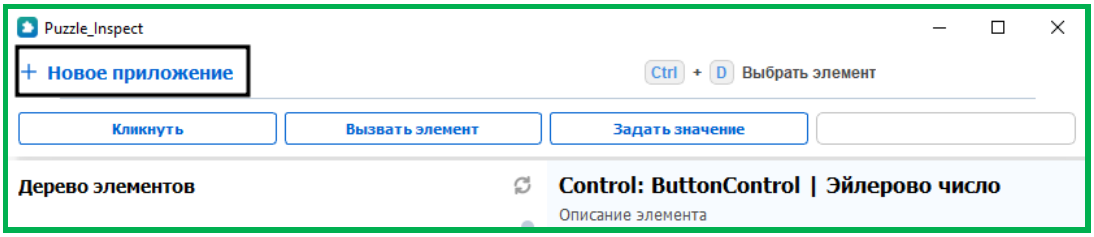


3 – Список атрибутов

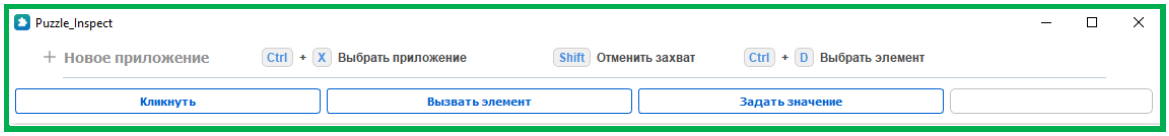
В правой части экрана расположены атрибуты выбранного элемента и соответствующие значения.

Для выбора другого приложения для просмотра дерева элементов требуется:

Выполнить клик на кнопку + Новое приложение;



Переключиться на новое приложение и нажать комбинацию клавиш Ctrl+X.

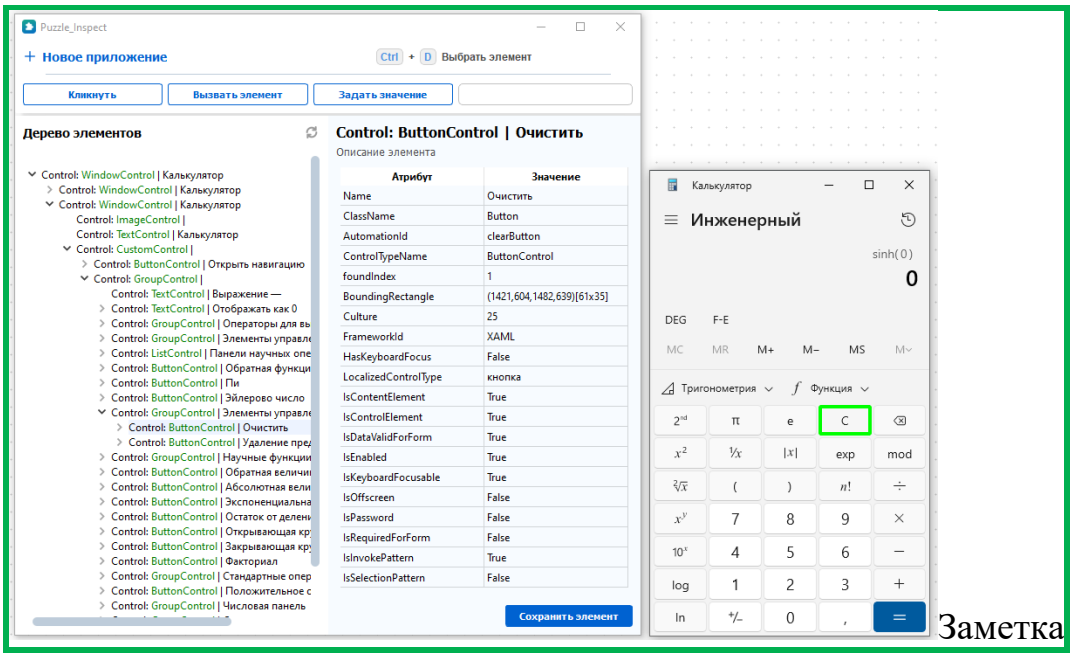


Для выбора элемента необходимо:

- Переключиться на выбранное приложение;

- Навести курсор мыши на интересующий элемент;
- Нажать комбинацию клавиш Ctrl+D;

Выбранный элемент будет выделен зеленым цветом и выделен в Puzzle инспекторе.

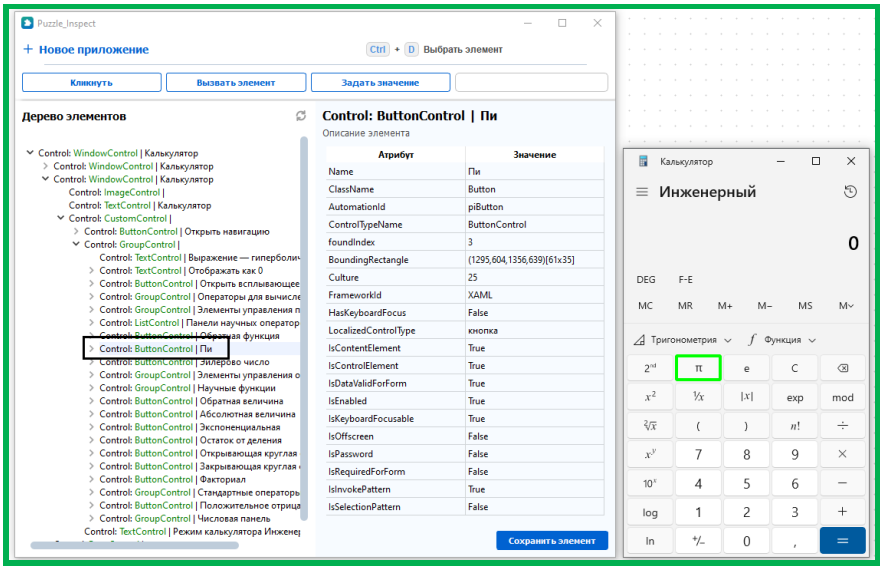


Заметка

Выбор элемента возможет только в текущем выбранном десктопном приложении.

Для выполнения клика на выбранный элемент необходимо:

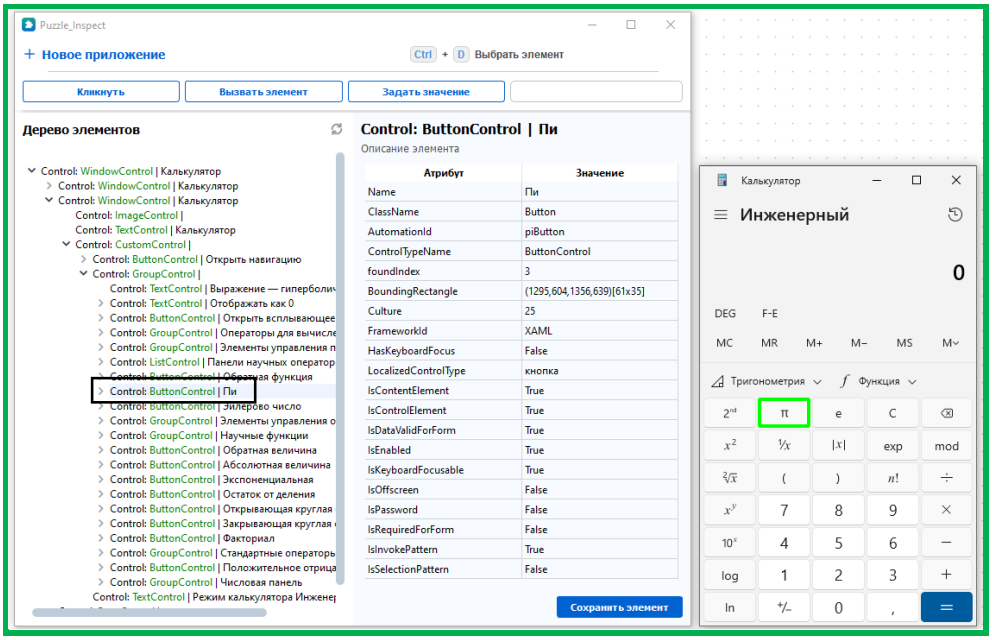
- Выбрать элемент (совершив клик) на панели Дерево элементов;



- Нажать на кнопку Кликнуть. Важно: действие не переносит окно десктопного приложения на передний план.

Для вызова выбранного элемента необходимо:

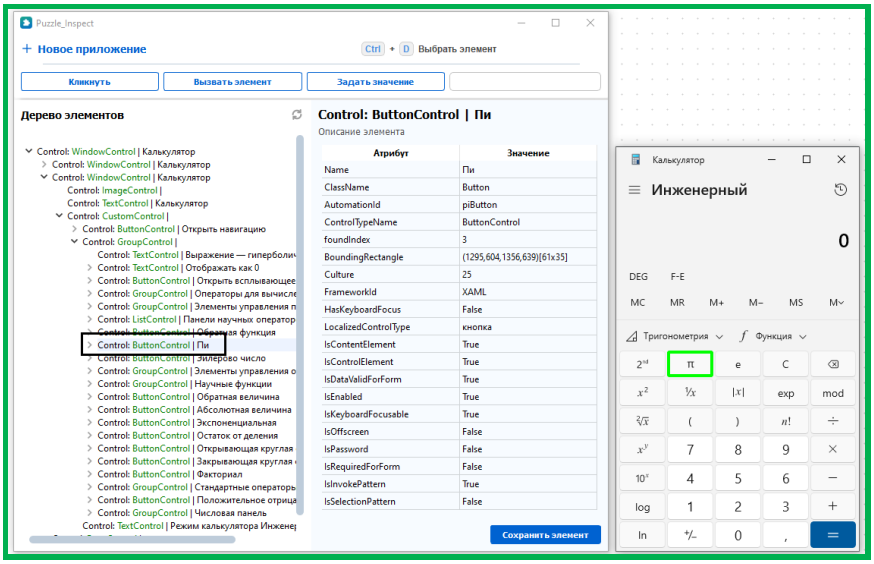
- Выбрать элемент (совершив клик) на панели Дерево элементов;



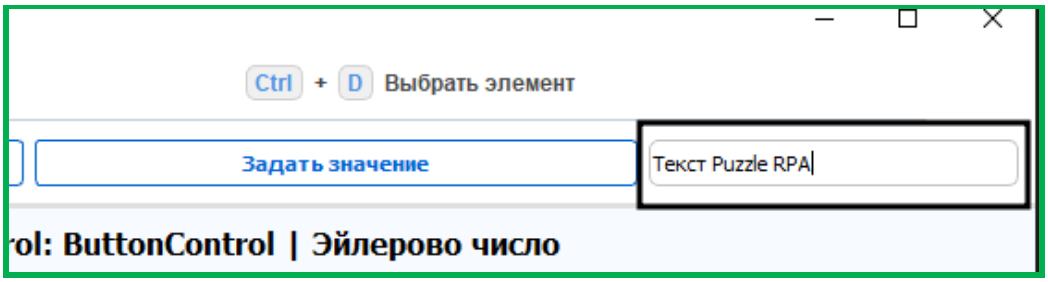
- Нажать на кнопку Вызвать элемент.

Если необходимо задать значение. Функция заполняет текстовое поле десктопного приложения. Для этого необходимо:

- Выбрать элемент (совершив клик) на панели Дерево элементов;



- Указать текст, который будет задан в элементе;

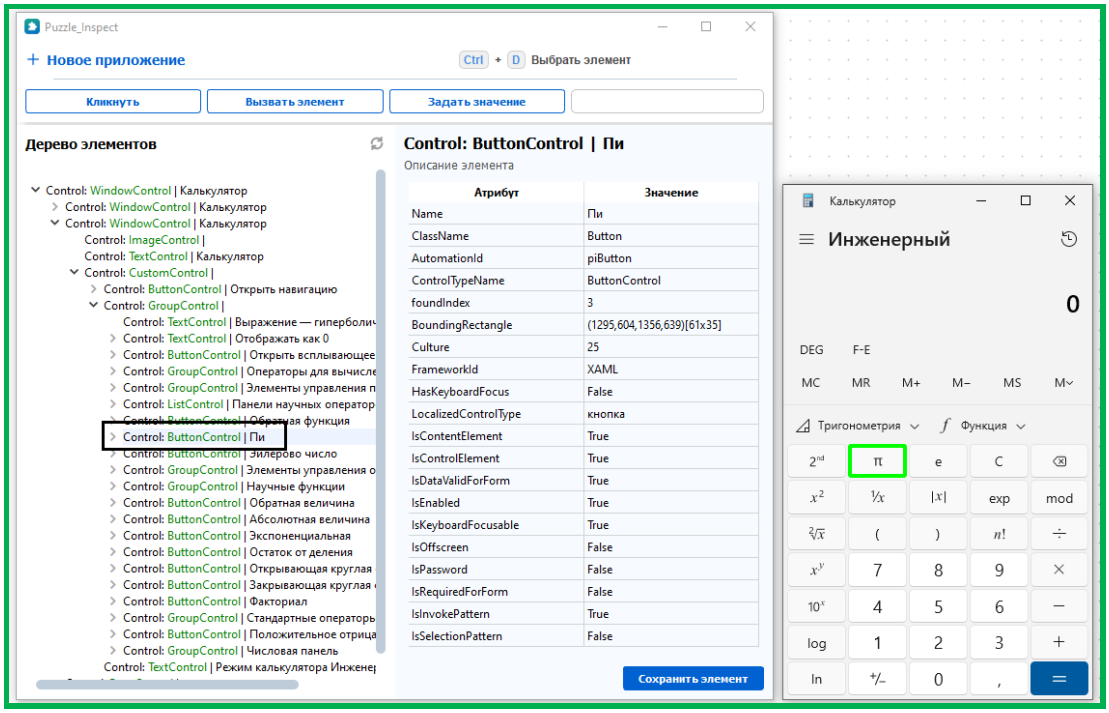


- Нажать на кнопку Задать значение.

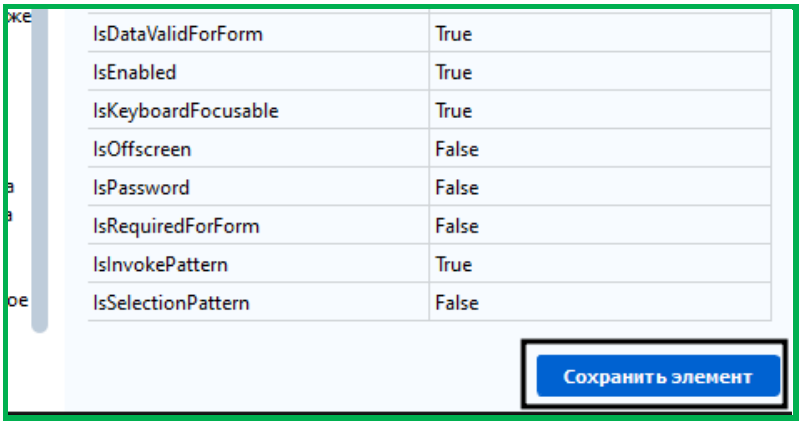
Важно: действие не переносит окно десктопного приложения на передний план.

Для сохранения выбранного элемента необходимо:

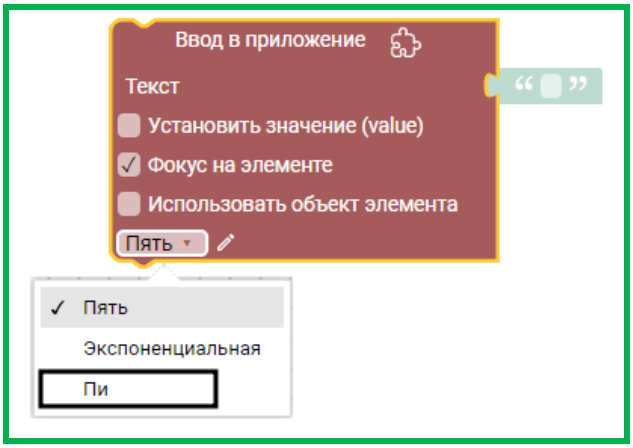
- Выбрать элемент (совершив клик) на панели Дерево элементов;



- Выполнить клик на кнопку Сохранить элемент в правом нижнем углу;



- Выбрать сохраненный элемент из выпадающего списка блока.



В совокупности Мастер UI и Puzzle инспектор образуют единую экосистему инструментов, которая существенно упрощает и ускоряет разработку роботов для десктопной автоматизации. Они позволяют разработчику точно определять элементы интерфейса, анализировать их свойства, отлаживать взаимодействие и эффективно строить надежные алгоритмы работы роботов с различными Windows-приложениями. Благодаря удобному интерфейсу, поддержке горячих клавиш и

настраиваемым параметрам, эти инструменты делают процесс разработки более прозрачным, контролируемым и продуктивным.

Ключевые блоки Desktop-автоматизации в Puzzle RPA

В процессе автоматизации десктопных приложений ключевую роль играет умение взаимодействовать с элементами пользовательского интерфейса так же, как это делает человек: открывать программы, нажимать кнопки, вводить текст и проверять результаты этих действий. Платформа Puzzle RPA предоставляет богатый набор инструментов для реализации такого взаимодействия.

В данной практической работе мы рассмотрим базовый набор операций, который составляет фундамент для создания любого робота, работающего с десктопными приложениями. Освоение этих блоков позволит вам автоматизировать типовые сценарии: от запуска программы и авторизации в ней до заполнения форм и получения результатов.

Что мы изучим:

- Открыть приложение – научимся запускать программы с указанием пути к исполняемому файлу.
- Активировать окно приложения – освоим перевод нужного окна на передний план для дальнейшей работы.
- Ожидать появления элемента – изучим механизмы синхронизации, чтобы робот не пытался взаимодействовать с еще не загрузившимся интерфейсом.
- Существует ли элемент – научимся проверять наличие элемента на экране для принятия решений в алгоритме.
- Поиск элементов – освоим базовый поиск одного элемента по заданным критериям.
- Прочитать значение элемента – узнаем, как извлекать текстовые данные из полей ввода, надписей и других элементов.
- Прочитать свойство элемента – научимся получать технические характеристики элементов (доступен ли элемент, виден ли он на экране и т.д.).
- Ввод в приложение – освоим заполнение текстовых полей.
- Клик по элементу – научимся имитировать нажатие кнопок, выбор пунктов меню и другие клики.
- Нажать горячие клавиши – изучим отправку сочетаний клавиш (например, Ctrl+C, Enter) для быстрого выполнения команд.

Понимание и уверенное применение этих десяти базовых блоков создаст прочную основу для перехода к более сложным сценариям автоматизации. Все рассмотренные операции будут выполняться с использованием встроенного инструмента Мастер UI, который позволяет точно идентифицировать элементы интерфейса и формировать корректные пути к ним.

В платформе Puzzle RPA все инструменты для автоматизации настольных программ сосредоточены в специальном разделе палитры компонентов – «Десктопные приложения». Именно здесь находится набор блоков, позволяющих робо-

ту взаимодействовать с любыми Windows-приложениями так же, как это делает человек.

Для удобства разработки все блоки в этом разделе логически сгруппированы по типу выполняемых действий. Это структурирование помогает быстро находить нужный инструмент и понимать, какие операции доступны для автоматизации. Выделяются три основные категории:

1. Интерфейс

Блоки этой категории отвечают за управление окнами приложений и их идентификацию. Сюда входят операции запуска и закрытия программ, активации окон, а также инструменты для настройки взаимодействия с приложениями.

2. Клавиатура

Эта группа объединяет блоки, имитирующие действия с клавиатуры. Здесь вы найдете инструменты для ввода текста в поля, отправки сочетаний горячих клавиш (например, Ctrl+C, Enter) и управления фокусом ввода. Эти блоки позволяют передавать данные и команды через клавиатурный ввод.

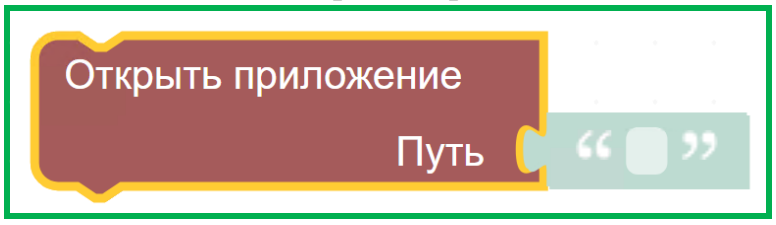
3. Мышь

В данной категории собраны блоки, эмулирующие движения и нажатия компьютерной мыши. Это не только простые клики по элементам, но и более сложные действия: клики со смещением, двойные клики, перетаскивание объектов (drag-and-drop), прокрутка колесика и работа с координатами курсора.

Такое разделение на группы «Интерфейс», «Клавиатура» и «Мышь» делает процесс разработки интуитивно понятным и позволяет быстро конструировать алгоритмы, комбинируя различные способы взаимодействия с пользовательским интерфейсом.

Рассмотрим подробно основные блоки.

1. Блок Открыть приложение



Блок открывает указанное приложение. Нам необходимо указать путь к файлу. Первый вариант – самый прямолинейный. Мы указываем полный, абсолютный путь к файлу. И второй вариант – все мы знаем про относительный путь – просто добавляем блок относительный путь и указываем название приложения.

2. Блок Активировать окно приложения



Блок «Активировать окно приложения» предназначен для перевода нужного окна на передний план и передачи ему фокуса ввода. Это критически важный блок, который гарантирует, что все последующие действия робота (клики, ввод текста, нажатие клавиш) будут направлены именно в то окно, с которым планируется взаимодействие. Без использования этого блока робот может отправить данные в случайное окно, оставшееся активным после предыдущих операций.

1. Функция «Использовать объект элемента»

Это мощная функция, которая позволяет:

- Подключить блок к переменной, содержащей объект элемента, полученный через Мастер UI или другие блоки
- Использовать JSON-строку с описанием пути к элементу для точной идентификации окна
- Обеспечить переиспользование уже настроенных элементов в разных частях сценария

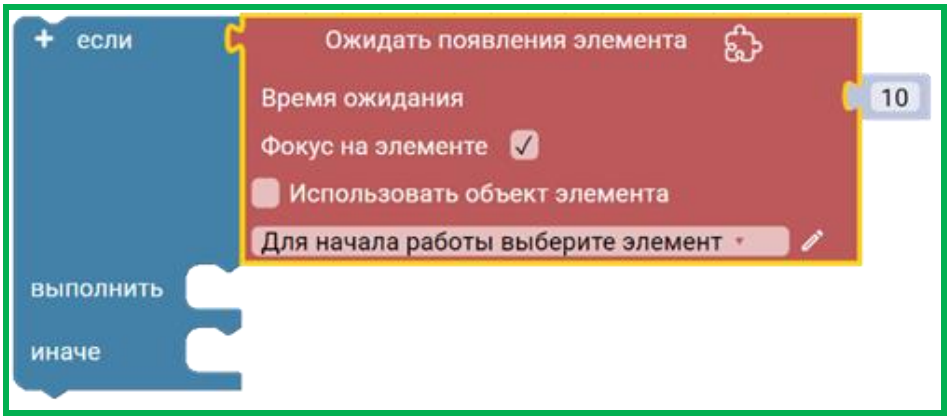
2. Поиск UI-элементов

Блок интегрирован с механизмом поиска элементов десктопных приложений. Это означает, что при настройке можно:

- Использовать Мастер UI для захвата окна
- Применить Puzzle инспектор для более детального анализа структуры окна
- Настроить сложные условия поиска, если простое совпадение по заголовку не подходит

Блок «Активировать окно приложения» – это своего рода система безопасности вашего робота. Он гарантирует, что каждое действие будет выполнено именно в том окне, для которого оно предназначено. Пренебрежение этим блоком – одна из самых частых причин нестабильной работы роботов в продуктивной среде. Используйте его всегда перед началом взаимодействия с приложением, и ваши сценарии станут надежнее и предсказуемее.

3. Блок «Ожидать появления элемента»



Блок «Ожидать появления элемента» предназначен для синхронизации работы робота с состоянием пользовательского интерфейса десктопного приложения. Это один из ключевых блоков, обеспечивающих стабильность автоматизации, поскольку он заставляет робота приостановить выполнение сценария до момента, когда указанный элемент интерфейса станет доступен на экране. Блок решает классическую проблему «робот работает быстрее, чем загружается интерфейс».

1. Гибкое управление временем ожидания

Параметр «Время ожидания» (таймаут) позволяет точно настроить, сколько секунд робот будет ждать появления элемента:

- Минимальное значение – для быстрых интерфейсов, где элемент появляется мгновенно
- Средние значения (5-10 секунд) – для типичных сценариев с обычной загрузкой
- Увеличенные значения (20-60 секунд) – для тяжелых приложений, работающих с большими объемами данных или медленной сетью

Важно: Если элемент не появится в течение указанного времени, блок сгенерирует ошибку, которую можно обработать в сценарии (например, повторить попытку или завершить процесс с уведомлением).

2. Опция «Фокус на элементе»

Этот параметр определяет, нужно ли дополнительно активировать элемент после его появления:

- Включено (True) – после обнаружения элемент будет выведен на передний план и получит фокус ввода. Полезно для полей ввода, кнопок и других интерактивных элементов
- Выключено (False) – элемент будет просто обнаружен, но фокус останется там, где был. Подходит для проверок существования или когда элемент не требует фокуса

3. Интеграция с объектной моделью

Параметр «Использовать объект элемента» открывает широкие возможности для переиспользования:

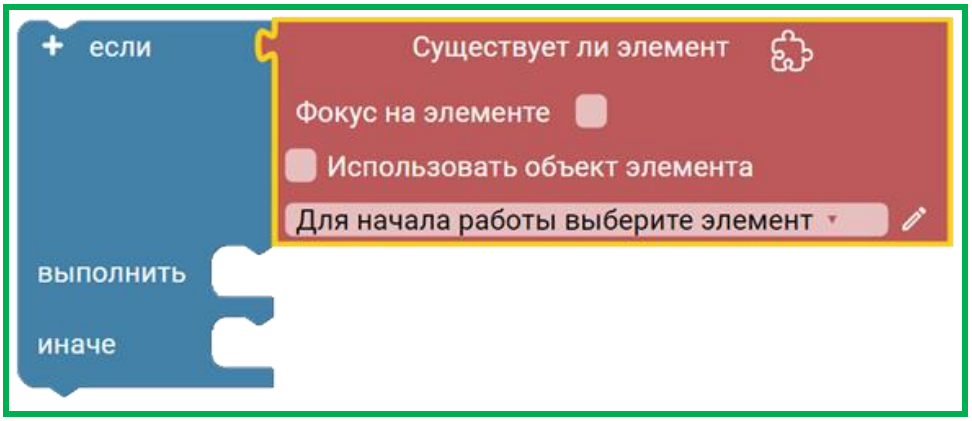
- Подключение переменной – можно передать ранее сохраненный объект элемента, полученный через другие блоки
- JSON-строка – возможность использовать сериализованное описание элемента для точной идентификации
- Динамическая подстановка – позволяет менять целевой элемент в зависимости от условий сценария

4. Работа с Мастер UI

Блок тесно интегрирован с инструментом Мастер UI для удобной настройки:

- Захват элемента – через интерфейс Мастера UI можно просто указать курсором на нужный элемент в приложении.
-
- Выпадающий список – все ранее сохраненные через Мастер UI элементы доступны для выбора из списка.
- Визуальное подтверждение – выбранный элемент можно подсветить для проверки корректности.

4. Блок «Существует ли элемент»



Блок «Существует ли элемент» выполняет мгновенную проверку наличия указанного UI-элемента в интерфейсе десктопного приложения. В отличие от блока ожидания, этот блок не приостанавливает выполнение сценария, а моментально возвращает результат проверки – True (элемент существует и виден на экране) или False (элемент отсутствует). Это идеальный инструмент для принятия решений в алгоритме робота.

1. Мгновенная проверка без ожидания

Главная особенность блока – отсутствие таймаута. Блок выполняется максимально быстро:

- Проверяет текущее состояние интерфейса здесь и сейчас
- Не ждет появления элемента, если его нет
- Не ждет исчезновения элемента, если он есть
- Мгновенно возвращает результат для дальнейшего принятия решения

2. Опция «Фокус на элементе»

Параметр позволяет дополнительно активировать элемент в случае, если он найден:

- Включено (True) – если элемент существует, он будет выведен на передний план и получит фокус ввода. Результат проверки всё равно вернется
- Выключено (False) – только проверка существования без изменения состояния интерфейса

Важно: Фокус устанавливается только если элемент найден. Если элемент отсутствует, никаких действий не производится.

3. Гибкая идентификация элементов

Блок поддерживает несколько способов указания целевого элемента:

- Через Мастер UI – интерактивный выбор элемента в приложении
- Из выпадающего списка – выбор ранее сохраненных элементов
- Объект элемента – использование переменной с объектом элемента
- JSON-строка – передача сериализованного описания элемента

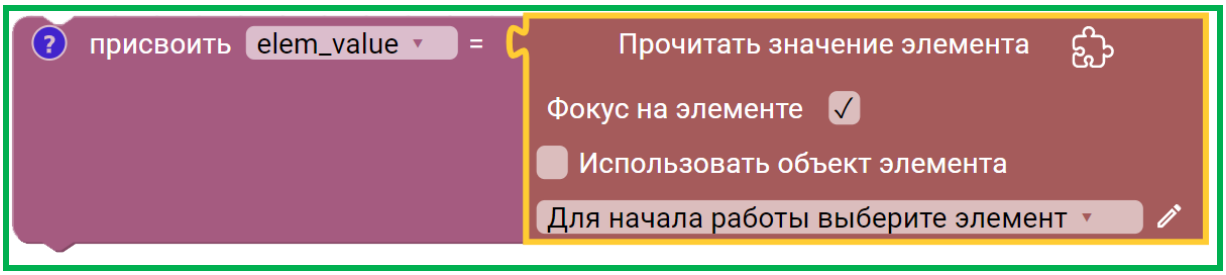
Преимущества использования:

- Скорость выполнения – блок не тратит время на ожидание
- Гибкость логики – позволяет строить сложные ветвления
- Обработка исключений – можно предвидеть проблемы до их возникновения

- Экономия ресурсов – не создает лишних нагрузок на систему
- Читаемость сценария – логика становится понятной и прозрачной

Блок «Существует ли элемент» – это глаза робота, которые мгновенно оценивают текущую ситуацию в интерфейсе. В сочетании с блоком «Ожидать появления элемента» они образуют мощный дуэт: первый отвечает за принятие решений, второй – за синхронизацию с приложением.

5. Блок «Прочитать значение элемента»



Блок «Прочитать значение элемента» предназначен для извлечения текстового содержимого из UI-элемента десктопного приложения. В отличие от блока «Прочитать свойство элемента», который может получать любые атрибуты, этот блок специализируется на чтении основного значения элемента – того, что пользователь обычно видит как текст или введенные данные. Чаще всего это значение атрибута Value (для полей ввода) или Name/Title (для надписей и кнопок).

Блок извлекает основное значение элемента, которое интерпретируется в зависимости от типа элемента:

Тип элемента	Что читается	Пример
Текстовое поле	Введенный текст	"Иванов Иван"
Надпись (Label)	Текст надписи	"Фамилия:"
Кнопка	Текст на кнопке	"Сохранить"
Заголовок окна	Текст заголовка	"Калькулятор"
CheckBox	Состояние (включен/выключен)	"True"/"False" или "Включено"
ComboBox (список)	Выбранное значение	"Москва"
Список (ListBox)	Выбранный элемент	"Пункт 1"

Опция «Фокус на элементе»

Параметр определяет, нужно ли активировать элемент перед чтением:

- Включено (True) – перед извлечением значения элемент получит фокус. Полезно для полей ввода, которые могут требовать активации для корректного отображения значения
- Выключено (False) – чтение происходит без изменения фокуса. Быстрее, но может не сработать для некоторых типов элементов

Особенность: Для некоторых приложений (особенно старых или написанных на специфических технологиях) значение может корректно читаться только после установки фокуса на элемент.

Параметр «Использовать объект элемента» предоставляет гибкие возможности:

- Переменная с объектом – подключение ранее сохраненного объекта элемента
- JSON-строка – передача сериализованного описания элемента в формате:

```
json
[{"ControlTypeName":"WindowControl","Name":"Калькулятор"}]
```

Блок полностью интегрирован с Мастер UI для удобного выбора элементов:

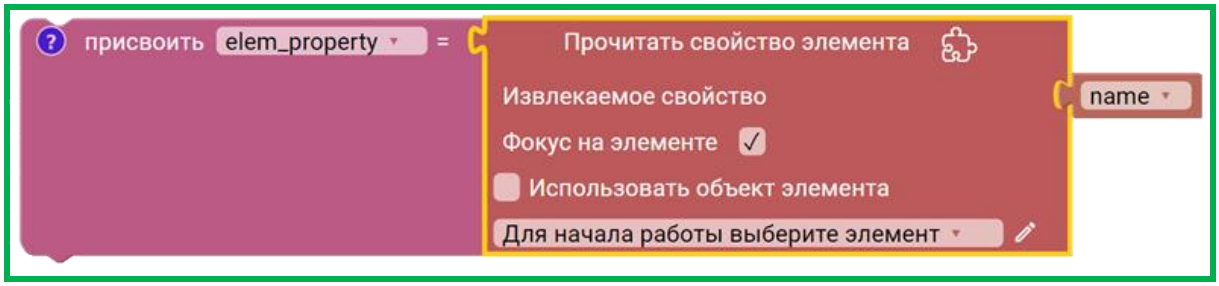
- Интерактивный захват – наведение на элемент и нажатие Ctrl+X
- Визуальное подтверждение – выбранный элемент подсвечивается зеленой рамкой
- Список сохраненных элементов – быстрый доступ к ранее настроенным элементам

Профессиональные советы:

- Всегда проверяйте, что читаете – используйте блок «Проверить» в Мастере UI
- Добавляйте ожидание перед чтением – особенно если данные только что изменились
- Обрабатывайте пустые значения – предусмотрите логику для случаев, когда элемент пуст
- Используйте фокус для сложных элементов – если чтение нестабильно, включите опцию фокуса
- Сохраняйте прочитанное в переменные – для дальнейшего использования в сценарии

Блок «Прочитать значение элемента» – это основной инструмент для получения данных из десктопных приложений. В паре с блоком «Ввод в приложение» он образует полный цикл обмена информацией: робот может и читать, и писать данные в интерфейс.

6. Блок «Прочитать свойство элемента»



Блок «Прочитать свойство элемента» – это мощный инструмент для получения технической информации об UI-элементе десктопного приложения. В отличие от блока «Прочитать значение», который извлекает только основное содержание элемента, этот блок позволяет получить доступ к любому атрибуту элемента: от его технических характеристик до состояния и координат на экране. Это своего рода «паспорт» элемента, содержащий всю доступную информацию о нем.

1. Широкий спектр извлекаемых свойств

Блок предоставляет доступ к десяткам различных свойств элемента. Рассмотрим их подробно по категориям:

- Идентификационные свойства (кто я?)

Свойство	Описание	Пример значения	Когда использовать
Name	Имя элемента, видимое пользователю	`"Сохранить", "OK", "Поле ввода"`	Для проверки текста на элементе
ControlType	Тип элемента управления	`"ButtonControl", "EditControl", "WindowControl"`	Для определения типа элемента
ClassName	Класс UI-элемента	`"Button", "Edit", "Static"`	Для технической идентификации
AutomationId	Уникальный идентификатор	`"btnSave", "txtName", "mainWindow"`	Самый надежный способ идентификации
FrameworkIdProperty	Технология реализации	`"Win32", "WPF", "WinForms", "Java"`	Для понимания особенностей элемента

- Свойства состояния (как я себя чувствую?)

Свойство	Описание	Пример значения	Когда использовать
`Активен (isEnabled)`	Доступен ли элемент для взаимодействия	`True/False`	Проверка, можно ли кликнуть
`Видимый (IsOffscreen)`	Отображается ли на экране	`True/False`	Проверка видимости (True = видим)
`IsOffscreenProperty`	Находится ли вне видимой области	`True/False`	True = элемент прокручен за пределы
`HasKeyboardFocus`	Имеет ли элемент фокус сейчас	`True/False`	Проверка текущего фокуса
`IsKeyboardFocusable`	Может ли получить фокус	`True/False`	Можно ли установить фокус
`IsSelected`	Выбран ли элемент	`True/False`	Для списков, чекбоксов
`IsPassword`	Является ли полем пароля	`True/False`	Определение типа поля

- Геометрические свойства (где я нахожусь?)

Свойство	Описание	Пример значения	Когда использовать
Координаты вершин (BoundingRectangle)	Прямоугольник элемента	`{"X":100,"Y":150,"Width":200,"Height":30}`	Для точного позиционирования

Центр элемента	Координаты центра	`{"X":200,"Y":165}`	Для кликов в центр
Размеры элемента	Ширина и высота	`{"Width":200,"Height":30}`	Для проверки размеров

- Структурные свойства (с кем я связан?)

Свойство	Описание	Пример значения	Когда использовать
Родитель элемента	Объект родителя	`[Object Element]`	Для навигации по иерархии
Дочерние элементы	Массив дочерних объектов	`[[Object Element], [Object Element]]`	Для обхода всех элементов

- Специализированные свойства

Свойство	Описание	Пример значения	Когда использовать
`NativeWindowHandleProperty`	Дескриптор окна Windows	`123456 (hex)`	Для низкоуровневых операций
`HelpText`	Всплывающая подсказка	`"Введите ваше имя"`	Для чтения подсказок
`LegacyIaccessibleRole`	Роль для accessibility	`"ROLE_SYSTEM_PUSHBUTTON"`	Для совместимости
`DefaultAction`	Действие по умолчанию	`"Нажать", "Выбрать"`	Понимание назначения
`IsDialog`	Является ли диалогом	`True/False`	Определение типа окна
`IsSelectionPattern`	Поддерживает ли выбор	`True/False`	Для сложных списков
`ExpandCollapseProperty`	Состояние сворачивания	`"Expanded"/"Collapsed"`	Для деревьев и групп

Особенность блока – возможность указать список свойств для одновременного извлечения:

В поле "Извлекаемое свойство" можно указать:

```
["Name", "ControlType", "IsEnabled", "BoundingRectangle"]
```

Блок вернет словарь (объект) со всеми запрошенными свойствами и их значениями. Это позволяет:

- Получить полную информацию об элементе за один вызов
- Снизить нагрузку на приложение (один запрос вместо нескольких)
- Упростить код и повысить его читаемость

Опция «Фокус на элементе»

Параметр определяет, нужно ли активировать элемент перед чтением свойств:

- Включено (True) – элемент получит фокус, что может быть необходимо для корректного чтения некоторых динамических свойств

- Выключено (False) – чтение без изменения фокуса, быстрее и безопаснее

Когда включать фокус:

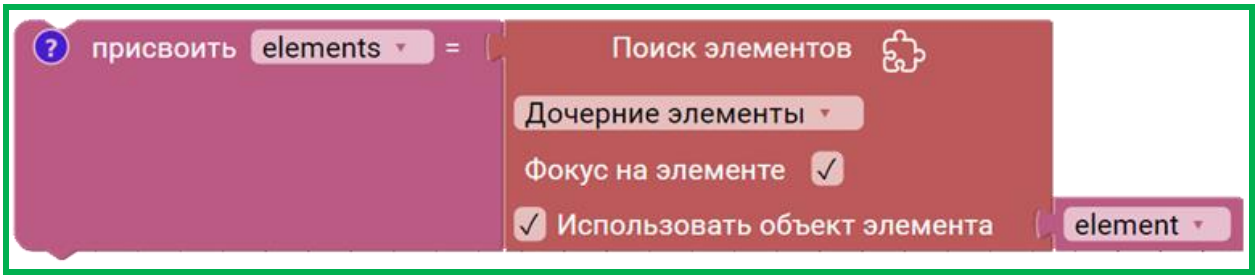
- Свойства, зависящие от состояния фокуса (HasKeyboardFocus)
- Элементы, которые обновляют свои свойства только при активации
- Проблемные приложения с нестандартной архитектурой

Параметр «Использовать объект элемента» предоставляет гибкие возможности:

- Переменная с объектом – подключение ранее сохраненного объекта
- JSON-строка – передача сериализованного описания элемента

В сочетании с другими блоками, «Прочитать свойство элемента» превращает робота из простого исполнителя в интеллектуального агента, способного анализировать ситуацию и принимать решения на основе реального состояния интерфейса.

7. Блок «Поиск элементов»



Блок «Поиск элементов» предназначен для поиска и получения коллекции UI-элементов десктопного приложения, соответствующих заданным критериям. В отличие от других блоков, которые работают с одним конкретным элементом, этот блок возвращает список (коллекцию) всех найденных элементов. Это открывает широкие возможности для обработки групп элементов: списков, таблиц, деревьев, всех кнопок на форме и т.д.

Важно понимать: Блок «Поиск элементов» – это не про один элемент, а про коллекцию. Он ищет на экране не первое попавшееся совпадение, а все элементы, которые соответствуют заданным критериям.

Результат его работы – это не одна переменная, а целый список (коллекция) элементов. И дальше мы можем с этим списком делать что угодно:

- Перебирать в цикле
- Считать количество
- Работать с каждым найденным по отдельности
- Фильтровать по дополнительным критериям
- Извлекать данные из каждого элемента

1. Параметр «Фокус на элементе»

Определяет, нужно ли активировать найденные элементы:

- Включен – каждый найденный элемент будет активирован (может быть полезно для динамических списков)
- Выключен – поиск происходит без изменения фокуса (быстрее и безопаснее)

2. Параметр «Использовать объект элемента» позволяет:

- Указать родительский элемент для поиска дочерних
- Использовать ранее сохраненный объект элемента
- Передавать JSON-описание элемента

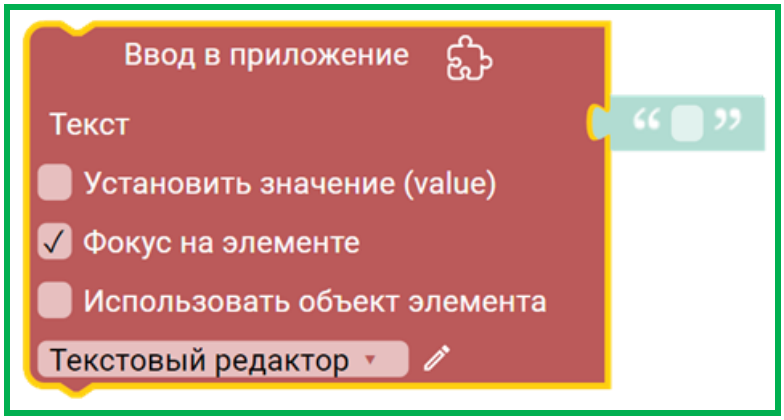
Блок «Поиск элементов» – это ключевой инструмент для работы с группами элементов в десктопных приложениях. Его главная сила в том, что он позволяет:

- Находить все элементы по заданным критериям
- Путешествовать по иерархии (вверх к родителям, вниз к детям)
- Обрабатывать коллекции в циклах
- Собирать массовые данные из таблиц и списков

Без этого блока автоматизация сложных интерфейсов с множеством однотипных элементов была бы невозможна. Именно он позволяет роботу работать с таблицами как с таблицами, со списками как со списками, а не с каждым элементом по отдельности.

В сочетании с циклами и условиями, блок «Поиск элементов» превращает робота в интеллектуального обработчика данных, способного анализировать и взаимодействовать с целыми структурами интерфейса, а не только с отдельными кнопками и полями.

8. Блок «Ввод в приложение»



Блок «Ввод в приложение» осуществляет интеллектуальную передачу текстовых данных в указанный UI-элемент десктопного приложения. Это не просто эмуляция нажатия клавиш, а полноценный механизм взаимодействия, который гарантирует, что текст попадет именно в то поле, которое нужно, и в тот момент, когда поле готово к приему информации.

1. Интеллектуальный механизм ввода

Блок выполняет последовательность действий «под капотом»:

- Поиск элемента – находит указанный элемент в интерфейсе
- Проверка готовности – убеждается, что элемент существует, видим и доступен
- Установка фокуса (если включено) – активирует элемент для ввода
- Очистка или замена текста – в зависимости от настроек
- Ввод текста – посимвольная или пакетная отправка
- Проверка результата (опционально) – может сверить введенное значение

2. Параметр «Текст»

Основной параметр блока – текст, который нужно ввести:

Тип данных: строка (string)

Пример: "Puzzle RPA", "Иванов Иван", "1234567890"

Особенности:

- Поддерживаются любые символы, включая спецсимволы и Unicode
- Можно использовать переменные для динамического ввода
- Поддерживаются многострочные тексты (с переносом строк)

3. Параметр «Установить значение (value)»

Этот чекбокс определяет способ ввода текста:

Режим	Что происходит	Когда использовать
Включен	Блок напрямую устанавливает свойство value элемента, минуя эмуляцию клавиатуры	Для полей, которые поддерживают программную установку значения. Быстрее и надежнее
Выключен	Блок эмулирует реальный ввод с клавиатуры (посимвольно)	Для полей с дополнительной логикой (маски, автодополнение, валидация на лету)

Преимущества установки value:

- Мгновенный ввод (весь текст появляется сразу)
- Не зависит от раскладки клавиатуры
- Работает даже если окно не в фокусе
- Обходит триггеры на посимвольный ввод

4. Параметр «Фокус на элементе»

Определяет, нужно ли активировать элемент перед вводом:

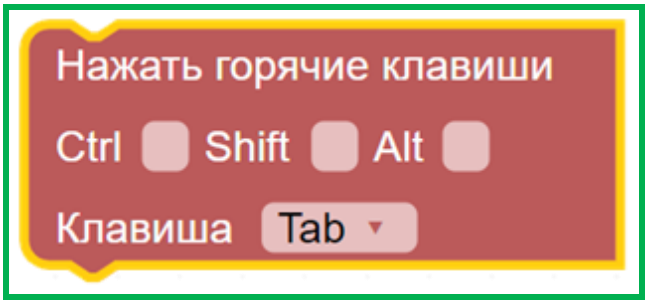
- Включен – перед вводом элемент получит фокус, будет активирован и выведен на передний план
- Выключен – ввод произойдет без изменения фокуса (работает только с режимом «Установить значение»)

5. Интеграция с объектной моделью

Параметр «Использовать объект элемента» предоставляет гибкие возможности:

- Переменная с объектом – подключение ранее сохраненного объекта элемента
- JSON-строка – передача описания элемента в текстовом формате

9. Блок «Нажать горячие клавиши»



Блок «Нажать горячие клавиши» эмулирует нажатие заданного сочетания клавиш на клавиатуре. Это универсальный инструмент для выполнения команд, которые в приложениях вызываются через клавиатурные сокращения: копирова-

ние, вставка, сохранение, переход между полями и многие другие действия. Блок позволяет отправлять как одиночные клавиши (Enter, Tab, Escape), так и сложные комбинации (Ctrl+C, Alt+F4, Ctrl+Shift+Esc). Блок содержит предустановленные популярные сочетания. Чекбоксы позволяют создавать любые комбинации клавиш.

Горячие клавиши отправляются в текущее активное окно. Поэтому перед использованием блока важно:

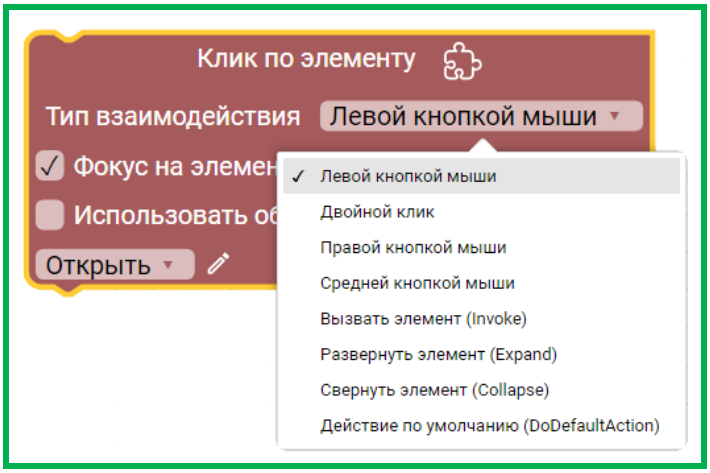
- Убедиться, что нужное окно активно (использовать блок «Активировать окно»)
- Или использовать блок «Фокус на элементе» перед вводом

Блок «Нажать горячие клавиши» – это мощный инструмент для быстрого выполнения команд в любых приложениях. Его главные преимущества:

- Независимость от UI – работает даже когда элементы интерфейса недоступны
- Скорость – команды выполняются мгновенно
- Универсальность – одни и те же комбинации работают в разных приложениях
- Простота – не нужно искать элементы в интерфейсе

В сочетании с блоками активации окон и ввода текста, этот блок позволяет эффективно управлять любыми приложениями, используя стандартные клавиатурные сокращения, к которым привыкли пользователи. Знание популярных комбинаций и умение их применять – важный навык разработчика RPA, который делает роботов быстрее и надежнее.

10. Блок «Клик по элементу»



Блок «Клик по элементу» выполняет программное нажатие на указанный UI-элемент десктопного приложения. Это один из самых часто используемых блоков в автоматизации, поскольку клики – основное действие пользователя при работе с любым интерфейсом. Блок имитирует различные типы нажатий: от простого левого клика до вызова специальных методов элемента.

1. Множество типов взаимодействия

Блок предоставляет богатый выбор способов взаимодействия с элементом:

- Стандартные клики мышью

Тип взаимодействия	Описание	Когда использовать
Клик левой кнопкой мыши	Обычный щелчок левой кнопкой (по умолчанию)	Нажатие кнопок, выбор пунктов меню, установка фокуса в поле
Двойной клик	Два быстрых нажатия левой кнопкой	Открытие файлов, выделение слова, активация ячеек в Excel
Правой кнопкой мыши	Щелчок правой кнопкой	Вызов контекстного меню, открытие свойств
Средней кнопкой мыши	Нажатие колесика	Открытие ссылок в новой вкладке, закрытие вкладок

- Специальные действия (без мыши)

Тип взаимодействия	Описание	Когда использовать
Вызвать элемент (Invoke)	Программный вызов действия элемента без эмуляции мыши	Для кнопок, ссылок, пунктов меню. Быстрее и надежнее мышиного клика
Развернуть элемент (Expand)	Раскрытие свернутого элемента	Для узлов дерева, выпадающих списков, групп
Свернуть элемент (Collapse)	Сворачивание развернутого элемента	Для скрытия дочерних элементов в дереве
Действие по умолчанию (DoDefaultAction)	Выполнение основного действия, заложенного разработчиком	Универсальный способ – что бы элемент ни делал по умолчанию

2. Сравнение режимов клика и Invoke

Важно понимать разницу между физическим кликом и программным вызовом:

Характеристика	Клик мышью	Invoke (Вызвать элемент)
Как работает	Эмулирует движение мыши и нажатие	Программно вызывает действие элемента
Скорость	Медленнее (нужно позиционирование)	Мгновенно
Надежность	Зависит от видимости и доступности	Высокая, если элемент существует
Требования	Элемент должен быть видим и доступен	Элемент должен быть доступен программно
Побочные эффекты	Может вызвать hover-эффекты	Только основное действие
Когда использовать	Когда важен визуальный эффект или Invoke не работает	Для максимальной скорости и надежности

3. Параметр «Фокус на элементе»

Определяет, нужно ли активировать элемент перед кликом:

- Включен – перед кликом элемент получит фокус, будет активирован и выведен на передний план
- Выключен – клик произойдет без изменения фокуса (работает только для Invoke и если элемент уже видим)

4. Интеграция с объектной моделью

Параметр «Использовать объект элемента» предоставляет гибкие возможности:

- Переменная с объектом – подключение ранее сохраненного объекта элемента
- JSON-строка – передача описания элемента в текстовом формате

Блок «Клик по элементу» – это основной инструмент для управления интерфейсом десктопных приложений. Богатый выбор типов взаимодействия позволяет:

- Имитировать любые действия пользователя мышью
- Использовать программные методы для максимальной скорости
- Работать со сложными элементами (деревья, списки)
- Вызывать контекстные меню

Главное преимущество блока – универсальность. Будь то простая кнопка, сложное дерево или контекстное меню – для всего есть подходящий тип клика.

ПОДВЕДЕНИЕ ИТОГОВ ПРАКТИЧЕСКОЙ РАБОТЫ

Мы завершили первую часть нашего погружения в мир Desktop-автоматизации на платформе Puzzle RPA. За это время мы не просто познакомились с отдельными блоками – мы выстроили фундамент, на котором будет держаться всё ваше дальнейшее развитие как RPA-разработчика.

Давайте оглянемся назад и посмотрим, какой путь мы прошли.

Мы изучили 10 ключевых блоков, которые покрывают 90% всех задач, возникающих при автоматизации десктопных приложений:

Блок	Что даёт роботу
Открыть приложение	Возможность запустить программу
Активировать окно	Гарантию, что действия попадут в нужное окно
Ожидать появления элемента	Терпение – умение ждать готовности интерфейса
Существует ли элемент	Глаза – мгновенную оценку ситуации
Прочитать значение элемента	Умение читать то, что написано на экране
Прочитать свойство элемента	Глубокое понимание природы каждого элемента
Поиск элементов	Способность видеть не один элемент, а коллекции
Ввод в приложение	Руки – возможность заполнять поля
Клик по элементу	Указательный палец – нажимать кнопки
Нажать горячие клавиши	Быстрые команды – управлять приложением без мыши

Каждый из этих блоков мы разобрали детально, изучили их параметры, фишки и особенности. Но главное – мы поняли философию, стоящую за каждым действием.

Мастер UI и Puzzle Inspect: наши главные инструменты

Отдельно стоит отметить работу с Мастером UI и Puzzle Inspect. Мы научились:

- Захватывать элементы с помощью горячих клавиш (Ctrl+X, Shift)
- Анализировать дерево элементов и их атрибуты
- Настраивать узлы и атрибуты для точной идентификации
- Использовать переменные в качестве значений атрибутов
- Проверять правильность выбора элемента
- Сохранять элементы для повторного использования

Эти инструменты – наши глаза и руки при разработке. Без них любой сценарий Desktop-автоматизации превращается в гадание. С ними – в инженерную работу.

Мы заложили прочный фундамент. Теперь вы:

- Понимаете, как робот «видит» интерфейс – через свойства элементов, а не через картинку
- Знаете, какие свойства бывают и как их использовать для идентификации
- Умеете пользоваться Мастером UI и Puzzle Inspect
- Освоили 10 базовых блоков, покрывающих большинство сценариев
- Усвоили принципы, которые делают сценарии надёжными и поддерживаемыми

Впереди – следующая практика, где мы перейдём к усложнённым блокам и техникам:

- Работа с таблицами и множественными элементами
- Продвинутое взаимодействие (drag-and-drop, клики со смещением, прокрутка)
- Специфика Java-приложений
- И многие другие инструменты, которые превратят вас из начинающего разработчика в профессионала

Главные выводы, которые нужно запомнить:

- Desktop-автоматизация – это не «взлом», а имитация действий человека через интерфейс
- Робот «слепой» – он не видит картинку, а «ощупывает» элементы через их свойства
- Свойства элемента – это единственный способ для робота понять, что перед ним
- Селектор – это «фоторобот» элемента, составленный из свойств
- Синхронизация важнее скорости – лучше подождать лишнюю секунду, чем упасть с ошибкой
- Каждое приложение уникально – нет единого рецепта, есть только принципы

- Надёжность достигается через – правильные селекторы, синхронизацию и обработку исключений

Теперь вы не просто «знаете, где какие блоки лежат». Вы понимаете философию Desktop-автоматизации, её принципы и подходы. Вы научились думать как робот и смотреть на интерфейс глазами разработчика.

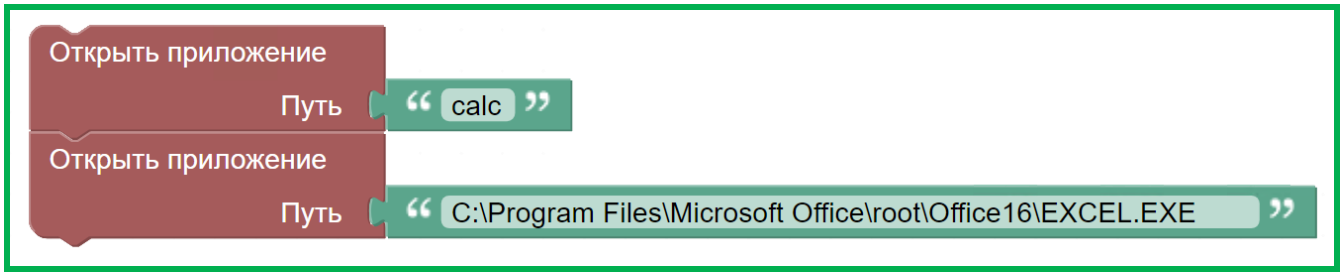
Этот фундамент останется с вами на всём пути развития в RPA. Будь то автоматизация старых бухгалтерских систем, современных CRM или веб-порталов – базовые принципы останутся неизменными.

ПРИМЕРЫ ЗАДАЧ

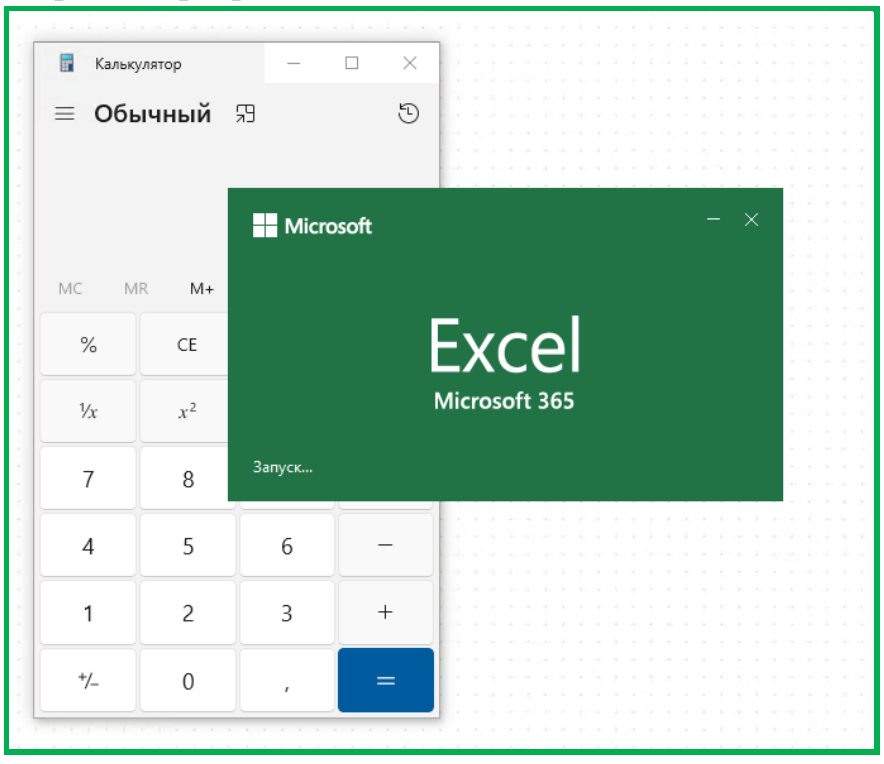
Пример 1 – Открыть приложение

В данном примере блок открывает калькулятор и приложение Excel:

1. Открывается калькулятор, приложение находится в стандартном окружении Windows, поэтому допускается не указывать полную директорию до исполнительного файла;
2. Открывается приложение Excel с использованием полной директории до исполнительного файла.



3. Результат: Открытое программное обеспечение.

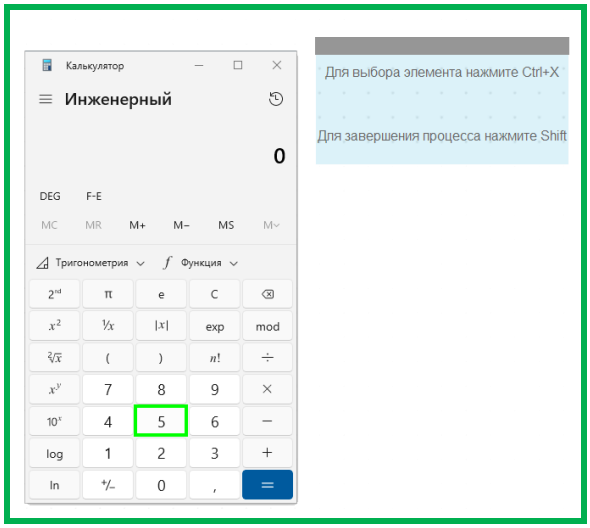


Пример 2 – Ожидать появление элемента

В данном примере открывается приложение **Калькулятор** и выполняется ожидание появления UI-элемента кнопки “5”.

Для выбора элемента в блоке **Ожидать появления элемента** используется инструмент Master UI для десктопных приложений. Для этого:

1. Открывается инструмент **Master UI для десктопных приложений**;
2. Выполняется выбор элемента, соответствующий кнопке 5 в приложение калькулятор.

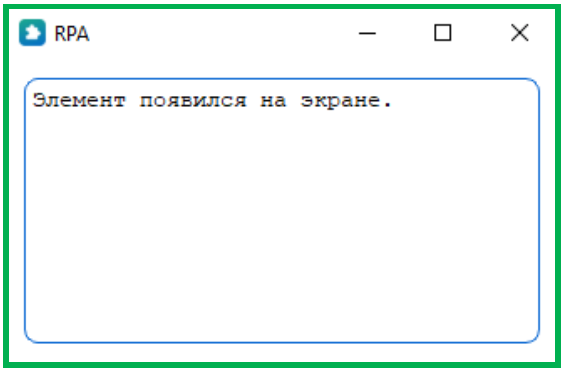


Алгоритм состоит из блоков:

- Открыть приложение **calc.exe**;
- Ожидать появления элемента;
- Уведомление пользователя с результатом ожидания появления UI-элемента.



Вывод результата на экран:

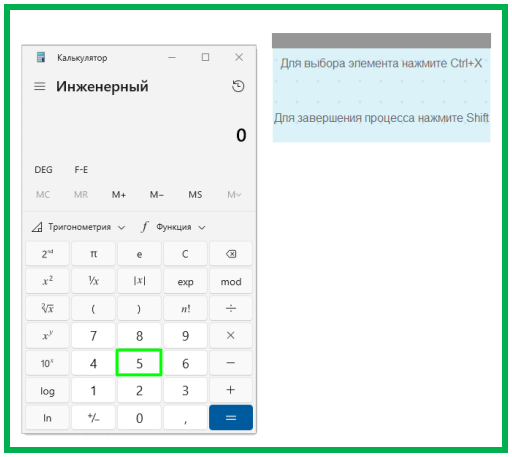


Пример 3 – Существует ли элемент

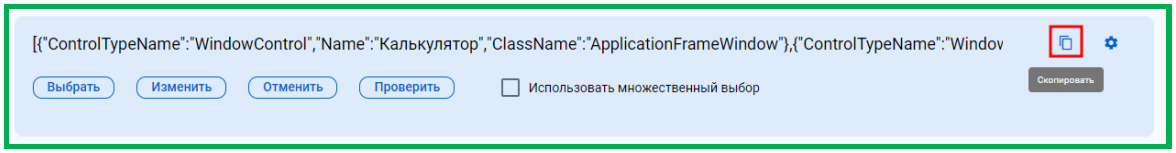
В данном примере открывается приложение **Калькулятор** и выполняется проверка существования UI-элемента кнопки “5”.

В качестве выбранного элемента в блоке **Существует ли элемент** используется JSON-строка. Для этого:

1. Открывается инструмент **Master UI для десктопных приложений**;
2. Выполняется выбор элемента, соответствующий кнопке 5 в приложение калькулятор.

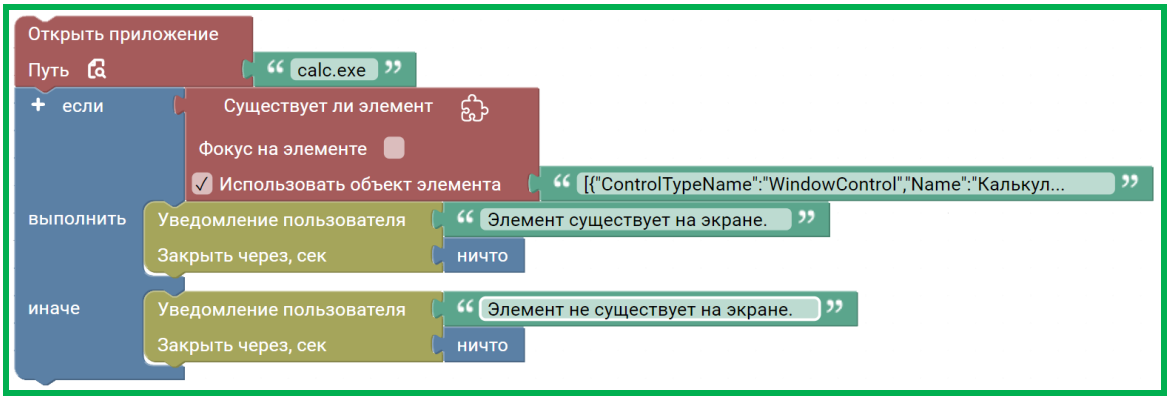


3. JSON-строка выбранного элемента копируется из интерфейса Master UI.

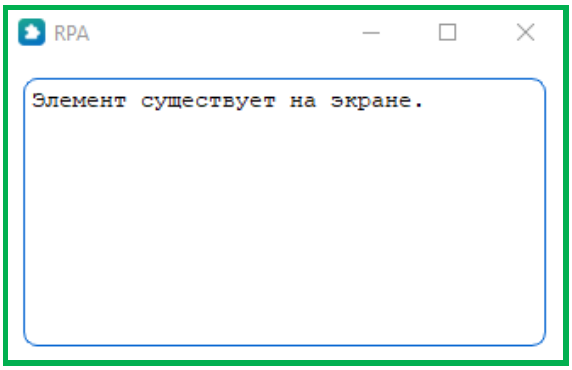


Алгоритм состоит из блоков:

- Открыть приложение **calc.exe**;
- Существует ли элемент;
- Уведомление пользователя с результатом проверки.



Вывод результата на экран:

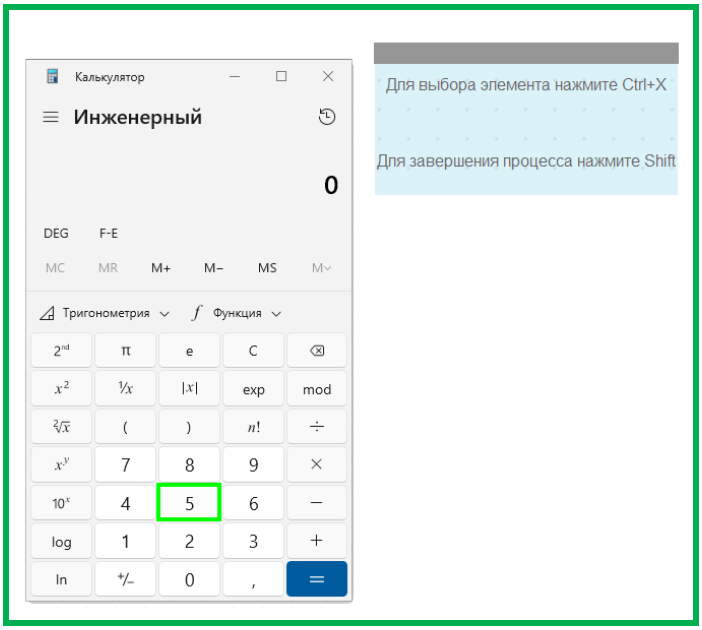


Пример 4 – Прочитать свойство элемента

В данном примере открывается приложение **Калькулятор** и считываются свойства **Name**, **ControlType** и **AutomationId** кнопки “5” в нем. Считанные свойства выводятся на экран.

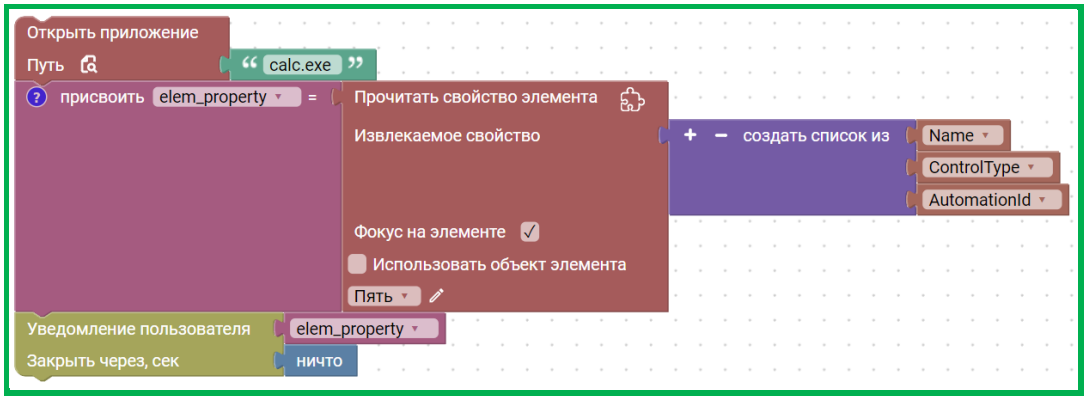
Для выбора элемента в блоке **Прочитать свойство элемента** используется инструмент Master UI для десктопных приложений. Для этого:

1. Открывается инструмент **Master UI для десктопных приложений**;
2. Выполняется выбор элемента, соответствующий кнопке 5 в приложение калькулятор.

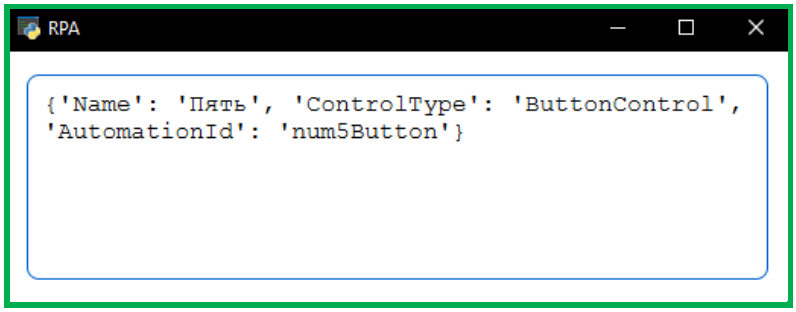


Алгоритм состоит из блоков:

- Открыть приложение **calc.exe**;
- Прочитать свойство элемента;
- Уведомление пользователя с результатом прочитанного свойства.



Вывод результата на экран:

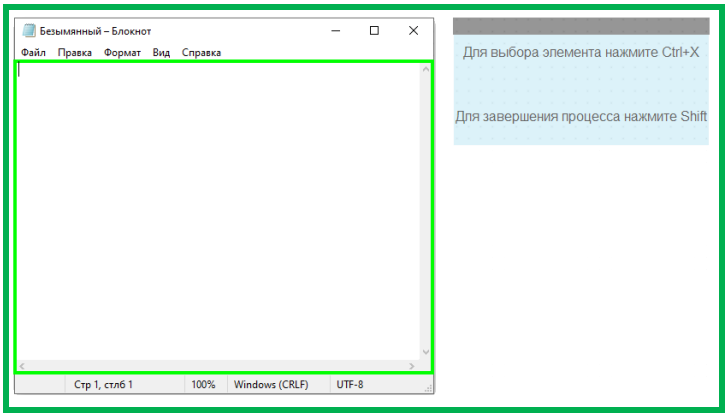


Пример 5 –Ввод в приложение

В данном примере открывается приложение **Блокнот** и в него записывается текст **Puzzle RPA**.

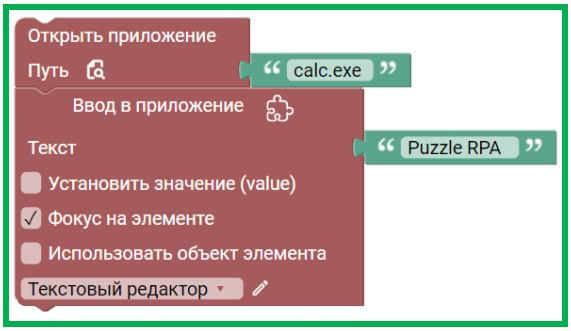
Для выбора элемента в блоке **Ввод в приложение** используется инструмент Master UI для десктопных приложений. Для этого:

1. Открывается инструмент **Master UI для десктопных приложений**;
2. Выполняется выбор элемента, соответствующий полю для ввода приложения.

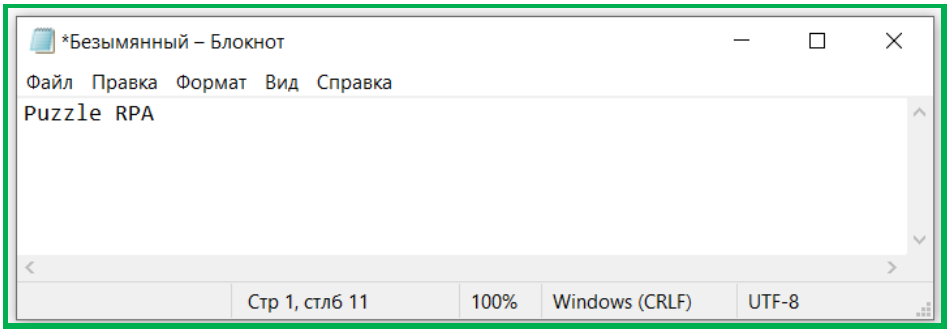


Алгоритм состоит из блоков:

- Открыть приложение **notepad.exe**;
- Ввод в приложение.

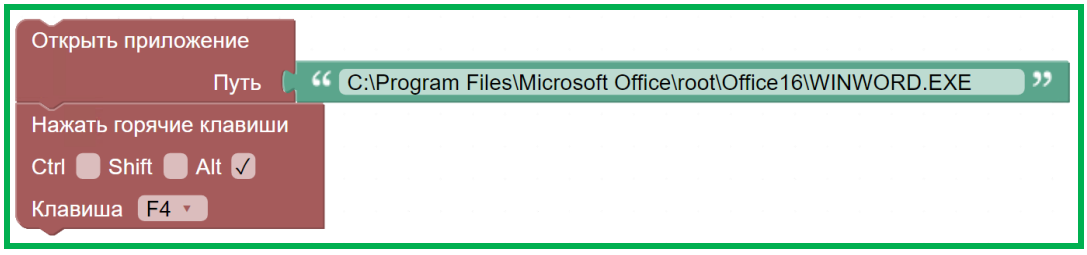


Результат ввода текста в **Блокнот**:



Пример 6 –Нажать горячие клавиши

В данном примере блок нажимает комбинацию клавиш Alt + F4 для выхода из открытого приложения Microsoft Word.

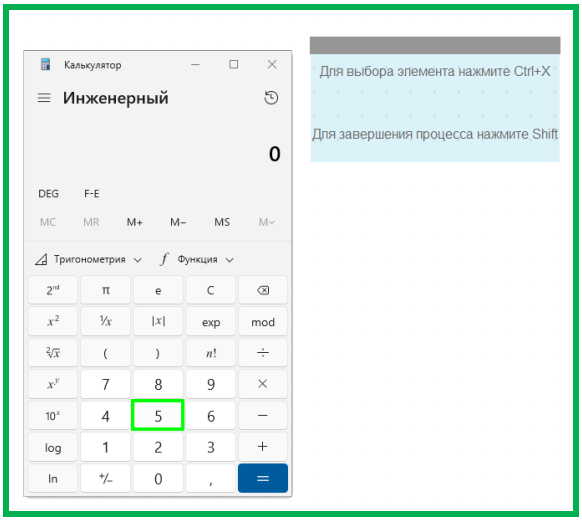


Пример 7 – Клик по элементу

В данном примере открывается приложение **Калькулятор** и выполняется клик по кнопке “5”.

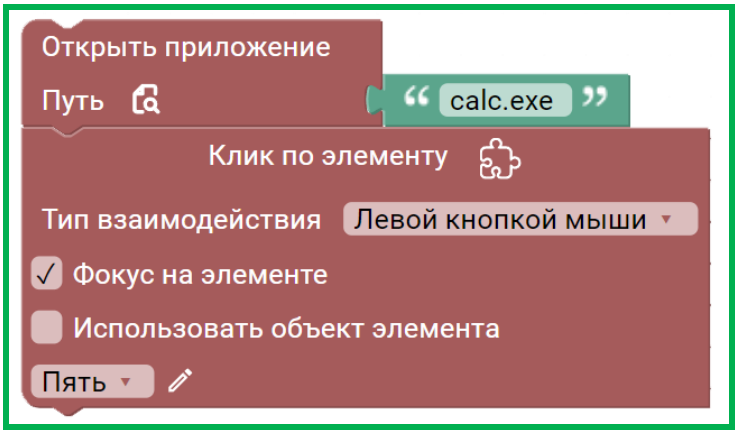
Для выбора элемента в блоке **Клик по элементу** используется инструмент Master UI для десктопных приложений. Для этого:

1. Открывается инструмент **Master UI для десктопных приложений**;
2. Выполняется выбор элемента, соответствующий кнопке 5 в приложение калькулятор.

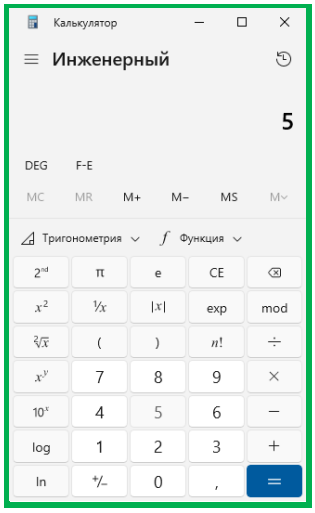


Алгоритм состоит из блоков:

- Открыть приложение **calc.exe**;
- Клик по элементу.



Результат: Приложение калькулятор после клика



Кейс 1. Прототип системы регистрации сервисных заявок

В компании TechFlow сотрудники регулярно создают внутренние сервисные заявки: запросы на техническую поддержку, доступ к корпоративным системам, настройку оборудования и другие рабочие задачи.

Сейчас сотрудники фиксируют такие обращения в разных таблицах и файлах, что приводит к дублированию информации и затрудняет контроль выполнения задач.

Чтобы упростить управление заявками, руководство компании решило создать простую настольную систему регистрации заявок, которая позволит сотрудникам быстро добавлять новые обращения и отслеживать их состояние.

Поскольку в будущем компания планирует автоматизировать часть операций с помощью программных роботов (RPA), разработчики решили сразу проверить, насколько удобно будет взаимодействовать с интерфейсом приложения средствами автоматизации.

В рамках данной практической работы необходимо создать прототип десктоп-приложения для регистрации заявок, а затем реализовать базовый сценарий автоматизации взаимодействия с этим приложением.

Задание 1. Разработка десктоп-приложения

Необходимо разработать простое настольное приложение с графическим пользовательским интерфейсом.

Технология разработки выбирается самостоятельно (например: Python Tkinter, C# WinForms, Java Swing или другие средства разработки).

Требования к интерфейсу приложения:

1. Приложение должно содержать форму регистрации сервисной заявки.
   - Поля ввода

     `Request Title` – тема заявки

     `Employee` – сотрудник, создавший заявку
   - Элемент выбора

     ✓ `Priority` – приоритет заявки

     Возможные значения:

     ✓ `Low`

     ✓ `Medium`

     ✓ `High`
   - Кнопки управления

     ➕ `Add Request` – добавление новой заявки

     🧹 `Clear Form` – очистка формы ввода
2. В нижней части окна должна находиться таблица зарегистрированных заявок.

   📊 Таблица заявок

ID	Title	Employee	Priority	Status

3. Статус новой заявки устанавливается автоматически:

```
{ "status": "New" }
```

4. При нажатии кнопки `Add Request` введённые данные должны добавляться в таблицу.
5. Управление активностью элементов интерфейса:

Для предотвращения создания некорректных записей необходимо реализовать управление доступностью элементов интерфейса.

- Кнопка `Add Request` должна быть неактивной (`IsEnabled = false`), если хотя бы одно обязательное поле формы не заполнено.
- После заполнения всех обязательных полей кнопка должна становиться активной (`IsEnabled = true`).
- Кнопка `Clear Form` должна оставаться активной независимо от состояния формы.

Задание 2. Базовая автоматизация работы с приложением

После разработки приложения необходимо создать процесс автоматизации в Puzzle RPA, который имитирует действия пользователя при создании заявок.

Алгоритм работы робота

1. Запустить разработанное приложение.
2. Дождаться появления главного окна программы.
3. Последовательно заполнить форму регистрации заявки:
   - ввести тему заявки
   - указать сотрудника
   - выбрать приоритет
4. Проверить состояние кнопки `Add Request`.
   - если кнопка неактивна (`IsEnabled = false`), робот должен определить, какое поле не заполнено, и дополнить данные;
   - после того как кнопка станет активной (`IsEnabled = true`), робот должен нажать её.
5. Добавить в систему не менее пяти заявок.
6. Проверить появление новых записей в таблице.
7. Логирование выполнения процесса

Для отслеживания работы робота необходимо реализовать журнал выполнения процесса (логирование).

Лог может сохраняться:

- в текстовый файл или
- в файл формата JSON.

В журнале должны фиксироваться:

- запуск приложения
- ввод данных в форму
- добавление заявки
- возможные ошибки
- завершение работы процесса

Пример записи журнала:

```
{
  "time": "2026-03-15 10:21:04",
  "action": "AddRequest",
  "title": "Access to CRM",
  "result": "success"
}
```

8. Модульность процесса

Для повышения читаемости и повторного использования кода рекомендуется выделять повторяющиеся действия в отдельные функции или подпроцессы:

- функция запуска приложения
- функция заполнения формы заявки
- функция добавления записи
- функция проверки состояния кнопки

Использование функций позволяет сделать сценарий автоматизации более структурированным и удобным для сопровождения.

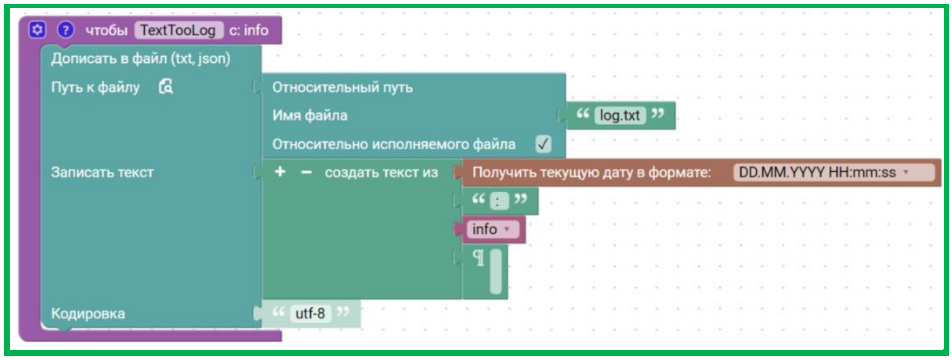
Реализация прототипа системы регистрации сервисных заявок

У нас есть простое, но важное задание: автоматизировать регистрацию заявок. Мы будем двигаться последовательно:

1. Запустим приложение
2. Прочитаем Excel-файл с данными
3. Добавим 5 заявок в систему
4. Проверим, что они появились

Всё будем делать модульно – каждая функция отвечает за свою задачу. Это позволит нам легко изменять и дополнять робота в будущем.

Функция логирования: `TextTooLog`



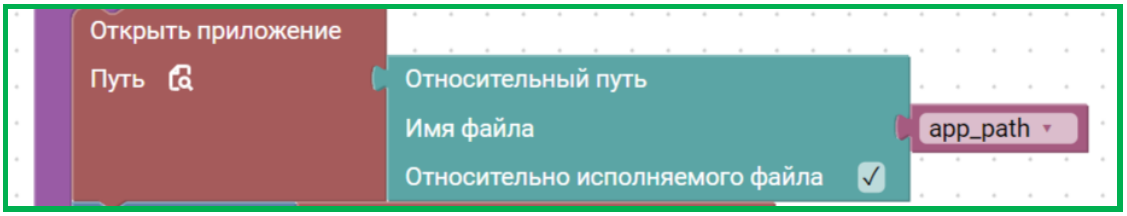
Шаг 1. Блок Получить текущую дату – Форматируем время: дд.мм.гггг чч:мм:сс

Шаг 2. Собрать строку – Склеиваем: [дата]: [info] + перевод строки

Шаг 3. Дописать в файл – Сохраняем в log.txt (в папке с программой)

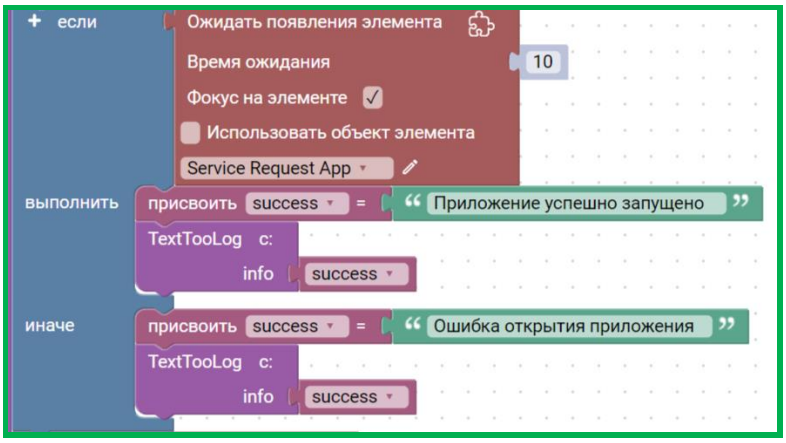
Функция запуска приложения: `start_exe`

Шаг 1. Блок Открыть приложение – Запускаем .exe файл по указанному пути



Шаг 2. Блок Ожидать появления элемента

- Ждём, пока появится окно "Service Request App" (таймаут 10 сек)
- Устанавливаем фокус на элементе



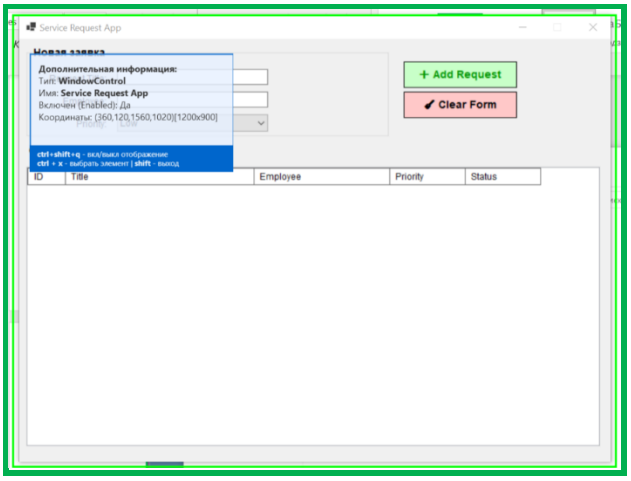
- Выбрать элемент через Мастер UI

✓ В окне Master UI нажмите кнопку "Выбрать"

✓ Наведите курсор мыши на главное окно приложения "Service Request App"

✓ Нажмите комбинацию клавиш Ctrl + X (по умолчанию) – элемент подсветится зелёной рамкой

✓ Для подтверждения выбора нажмите клавишу Shift



Шаг 3.

- Если окно появилось:

✓ Устанавливаем `success = "Приложение успешно запущено"`

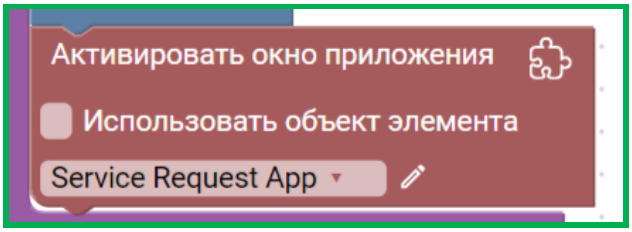
✓ Логируем успех через `TextTooLog`

- Если окно НЕ появилось:

✓ Устанавливаем `success = "Ошибка открытия приложения"`

✓ Логируем ошибку

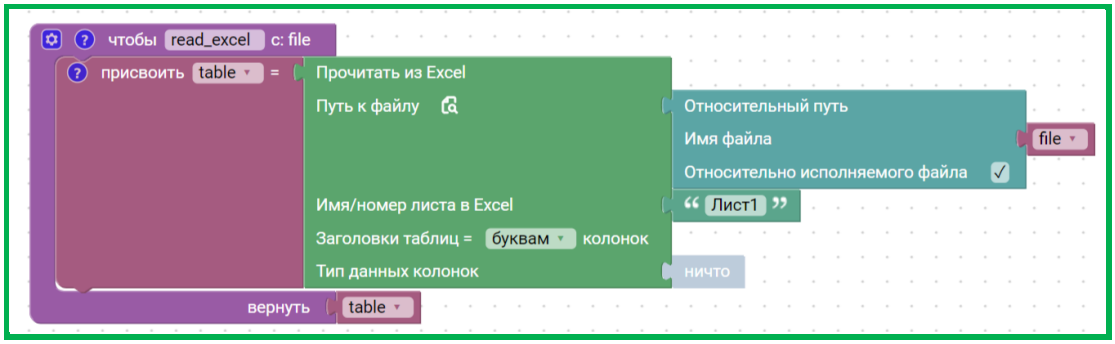
Шаг 4. Блок Активировать окно приложения – Устанавливаем фокус на окно приложения



Функция чтение Excel: `read_excel`

Шаг 1. Блок Прочитать из Excel

- Читаем файл requests.xlsx, лист "Лист1"
- Сохраняем все данные в переменную table



Шаг 2. Вернуть результат – Возвращаем `table` в основной процесс

Функция 3: `fill_form` (заполнение и добавление заявок)

Шаг 1. Блок Цикл – Повторяем 5 раз (для i от 1 до 5)

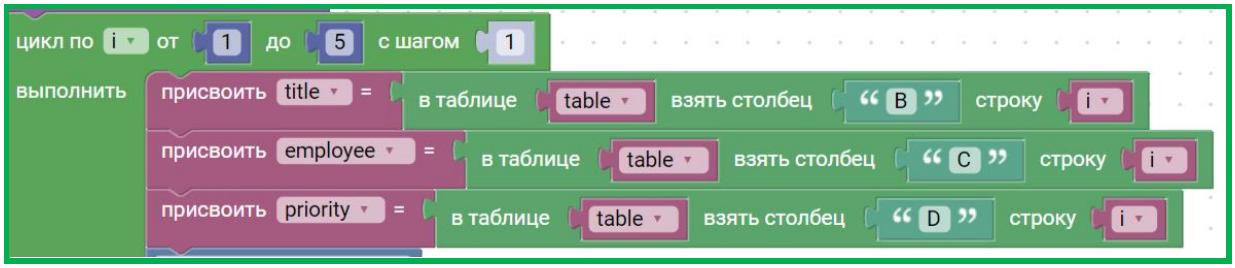
Шаг 2. Внутри цикла для каждой заявки – Получить данные из Excel

Из таблицы `table` берём:

✓ Колонка B → `title` (тема заявки)

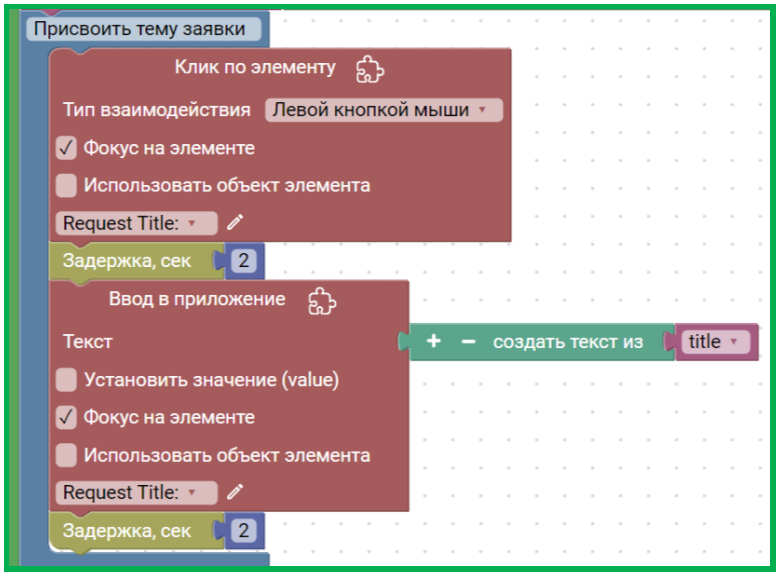
✓ Колонка C → `employee` (сотрудник)

✓ Колонка D → `priority` (приоритет)



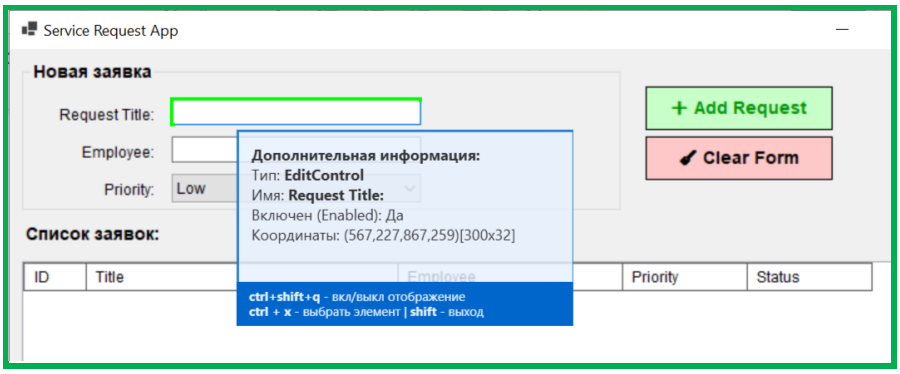
Шаг 3. Заполнить Тему заявки

- Клик по полю `Request Title`
- Задержка 2 сек
- Ввод текста из переменной `title`



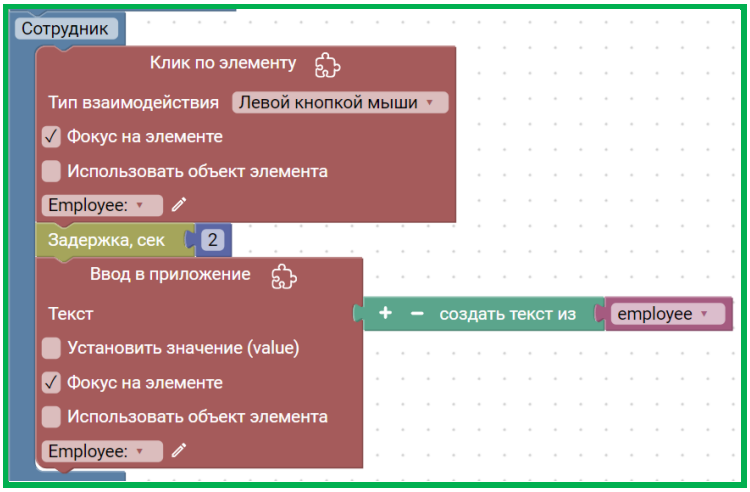
- Выбрать элемент через Мастер UI (поле `Request Title`)

✓ В окне Master UI нажмите кнопку "Выбрать"
✓ Наведите курсор мыши на поле Request Title в вашем приложении
✓ Нажмите комбинацию клавиш Ctrl + X – элемент подсветится зелёной рамкой
✓ Для подтверждения выбора нажмите клавишу Shift

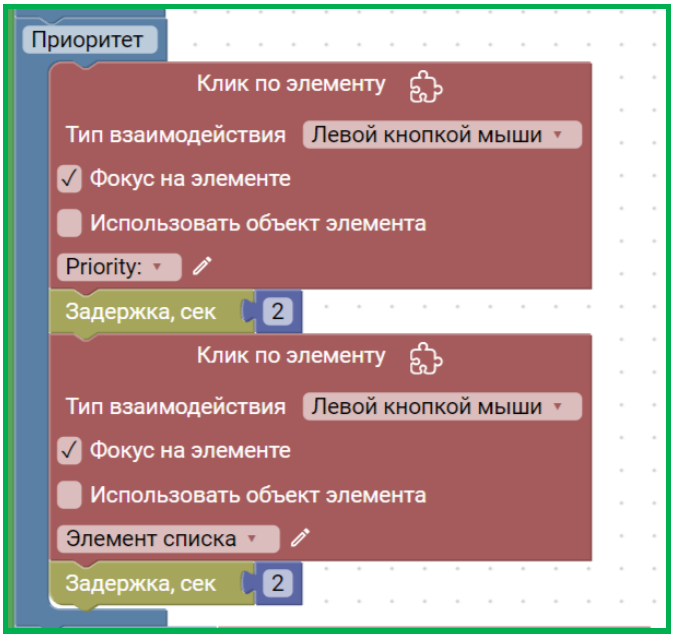


Шаг 4. Заполнить поле Сотрудник

- Клик по полю `Employee`
- Задержка 2 сек
- Ввод текста из переменной `employee`

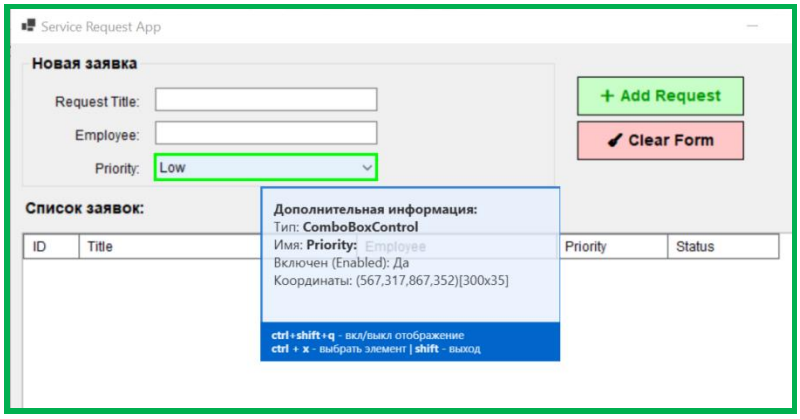


Шаг 5. Выбрать приоритет



- Клик по выпадающему списку Priority
- Выбрать элемент через Мастер UI (сам выпадающий список Priority)

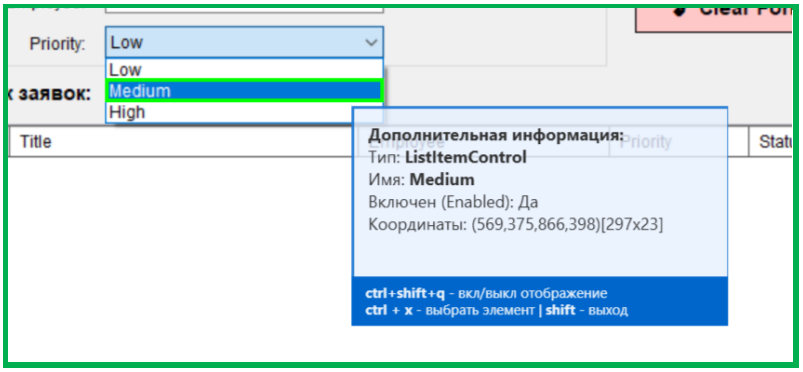
✓ В окне Master UI нажмите кнопку "Выбрать"
✓ Наведите курсор мыши на поле Priority
✓ Нажмите комбинацию клавиш Ctrl + X (по умолчанию) – элемент подсветится зелёной рамкой
✓ Для подтверждения выбора нажмите клавишу Shift



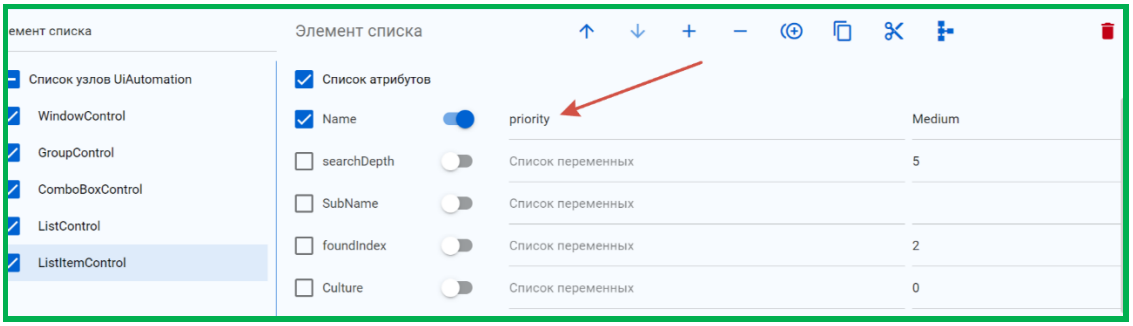
- Задержка 2 сек
- Клик по нужному пункту списка (`Low`/`Medium`/`High`)
- Пошаговая работа с Master UI для выбора пункта списка

Подготовка – раскрыть список вручную. Важный момент! Чтобы Master UI мог "увидеть" пункты списка, список должен быть раскрыт в момент выбора элемента.

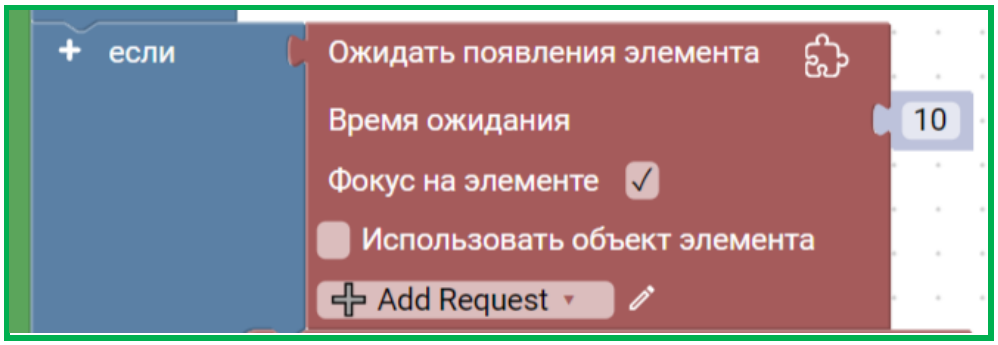
✓ Временно сделайте это вручную: кликните по полю `Priority` в вашем приложении, чтобы список раскрылся
✓ Убедитесь, что пункты `Low`, `Medium`, `High` видны на экране
✓ Не закрывайте список – переходите к Master UI



✓ После выбора в Master UI отобразится список узлов
✓ Обратите внимание на последний узел: именно здесь нам нужно, чтобы робот выбирал разные значения в зависимости от переменной `[priority]` (`Low`, `Medium`, `High`).

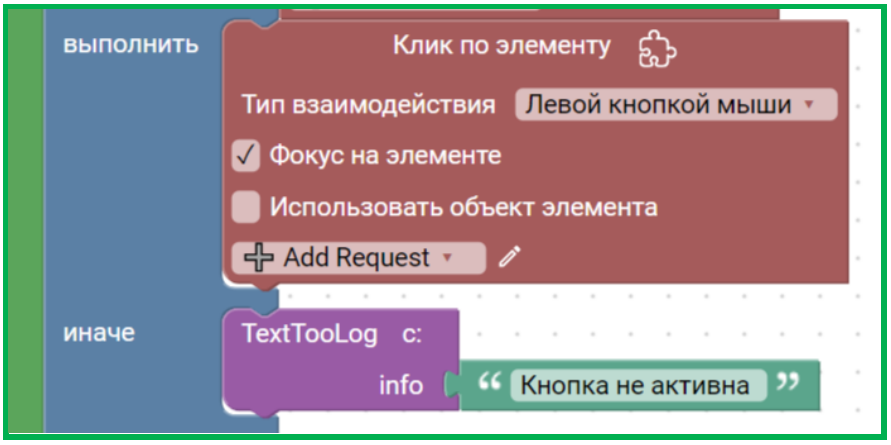


Шаг 6. Проверить кнопку `Add Request` – Ждём появления кнопки (таймаут 10 сек)

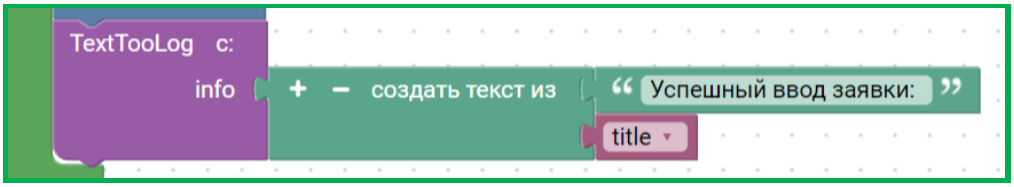


Шаг 7. Нажать Add Request

- Если кнопка появилась → кликаем
- Если нет → логируем "Кнопка не активна"



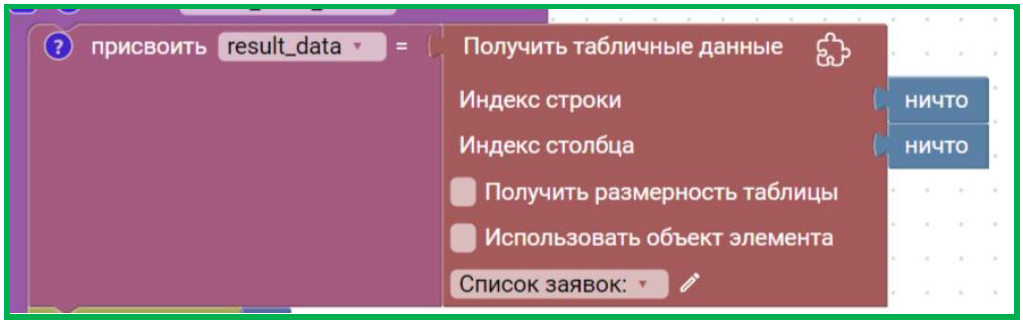
Шаг 8. Логирование – Записываем "Успешный ввод заявки: " + title (наименование заявки)



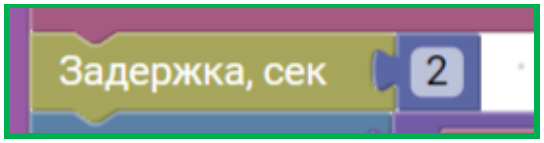
Функция проверка таблицы: check_table_rows

Шаг 1.Блок Получить табличные данные

- Читаем всю таблицу dataGridRequests
- Сохраняем в result_data



Шаг 2. Блок Задержка – 2 секунды (обязательна после чтения `dataGrid`)



Шаг 3. Блок Условие

- Если таблица НЕ прочиталась (`len(result_data) == 0`):

✓ Логируем "Не удалось прочитать таблицу"

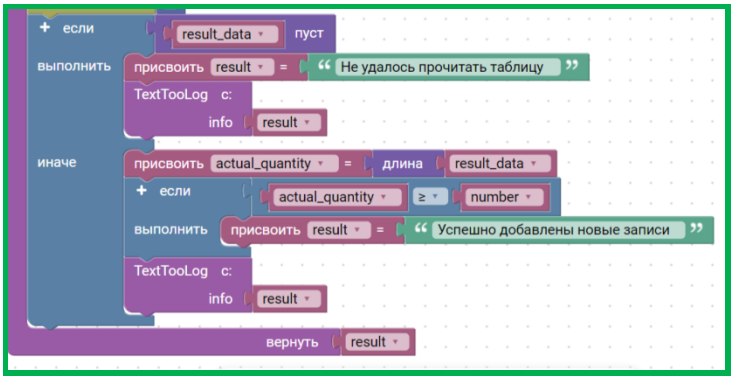
- Иначе (таблица прочиталась):

✓ Считаем количество строк actual_quantity = len(result_data)

✓ Сравниваем с ожидаемым числом (параметр number = 5)

✓ Если actual_quantity >= 5:

✓ Логируем "Успешно добавлены новые записи"



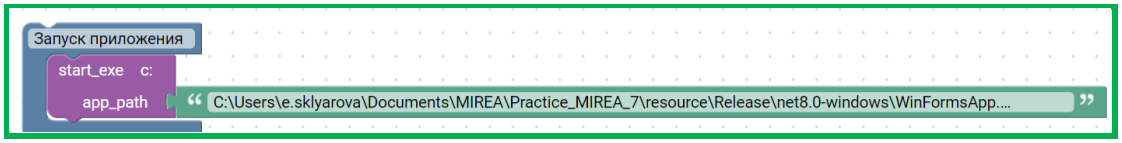
Шаг 4. Вернуть результат – Возвращаем `result` в основной процесс

Основной процесс (`main`)

Шаг 1. Запуск приложения

- Вызываем функцию start_exe
- Пишем полный путь к файлу exe.

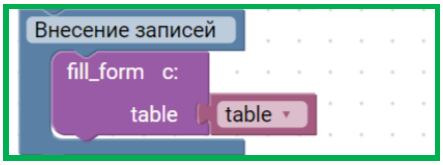
Приложения имеют зависимости – требует наличия других файлов в той же папке, то есть ожидает, что запуск будет из папки, где лежат все его компоненты



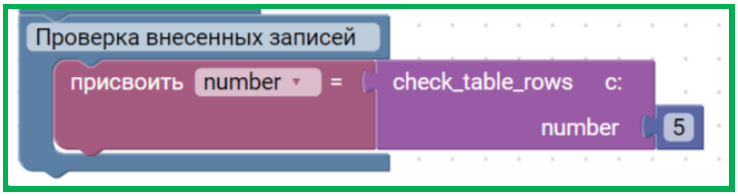
Шаг 2. Чтение Excel – Вызываем функцию `read_excel('requests.xlsx')` → получаем `table`



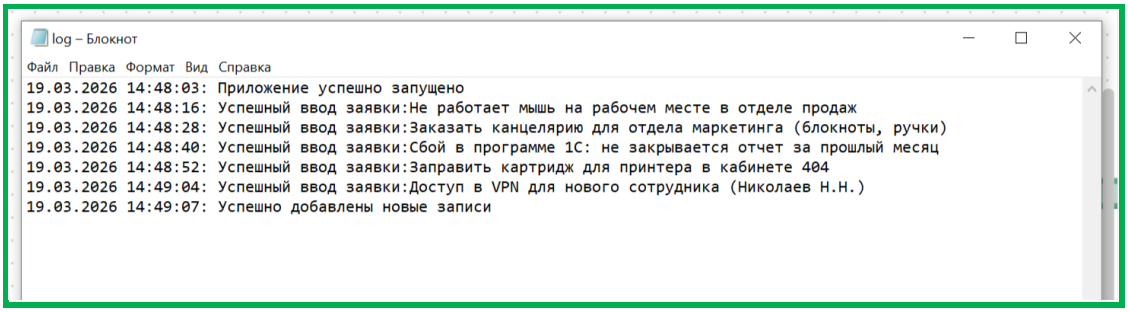
Шаг 3. Добавление заявок – Вызываем функцию `fill_form(table)` → добавляем 5 заявок



Шаг 4. Проверка таблицы – Вызываем `check_table_rows (5)` → проверяем, что добавилось 5 записей



Шаг 5. Проверяем лог файлы



Практическая работа выполнена в полном объёме. Разработано:

- Рабочее десктоп-приложение для регистрации заявок
- Модульный RPA-робот для автоматизации процесса
- Система логирования для контроля выполнения
- Проверка результатов автоматизации

Робот полностью заменяет ручной труд при вводе заявок, исключает ошибки и значительно ускоряет процесс.

КОНТРОЛЬНЫЕ ЗАДАНИЯ

Общая часть задания практической работы

1. Ознакомьтесь со структурой и материалом методических указаний.
2. Ознакомьтесь с теоретическим материалом, изложенным в разделе «Теоретическая часть».
3. В случае отсутствия установленного ПО «Студия Puzzle RPA», скачайте по ссылке инсталляционный файл https://cloud.puzzle-rpa.ru/index.php/s/N87hb5sE2Lm0EQp (пароль для загрузки - `GDJoL9Yz`), и установите программный продукт. Для авторизации в студии введите

Логин - `mirea-student`
Пароль - `EEasSdDNKK5m`

Обратите внимание, что логин и пароль для доступа к полной версии продукта – временные и действуют до 05.07.2026 г.

4. Внимательно изучите материал раздела «Теоретическая часть».
5. Сделайте полную копию раздела «Теоретическая часть» из методички и поместите её в начало отчета.
6. Внимательно изучите и разберите материал раздела «Примеры задач».
7. Внимательно изучите условие контрольного примера (аналогичное задание для уникальной предметной области будет предложено Вам для самостоятельного выполнения).
8. Просмотрите видео-урок по выполнению задания контрольного примера.
9. **Поместите в отчет условие задания своего варианта, скопировав его из учебного репозитория**.
10. Для выполнения Задания 1 Вашего варианта создайте в любой удобной для Вас IDE Windows-приложение под именем ***PR7_Surname_RPA_Desktop_p1_Problem1_varN***.

КОНТРОЛЬНЫЕ ЗАДАНИЯ

Контрольное задание 1

Задание 1. Разработка десктоп-приложения

Необходимо разработать настольное приложение с графическим пользовательским интерфейсом.

Технология разработки выбирается студентом самостоятельно (например: **C# WinForms, Java Swing** и др.).

🖥️ Требования к интерфейсу приложения

Приложение должно содержать форму регистрации доставки.

📝 Поля ввода

- **Package** — наименование посылки
- **Driver** — имя водителя

🎛️ Элемент выбора

- **Delivery Type** — тип доставки

Возможные значения:

- Standard
- Express
- Same-day

- **Priority** — приоритет доставки

Возможные значения:

- Low
- Medium
- High

🔘 Кнопки управления

- ➕ **Add** — добавление новой доставки
- 🧹 **Clear Form** — очистка формы

📊 Таблица доставок

В нижней части окна должна отображаться таблица зарегистрированных доставок со следующей структурой:

ID	Package	Driver	Delivery Type	Priority	Status

При добавлении новой записи статус устанавливается автоматически:

{ "status": "New" }

⚙️ Управление активностью элементов интерфейса

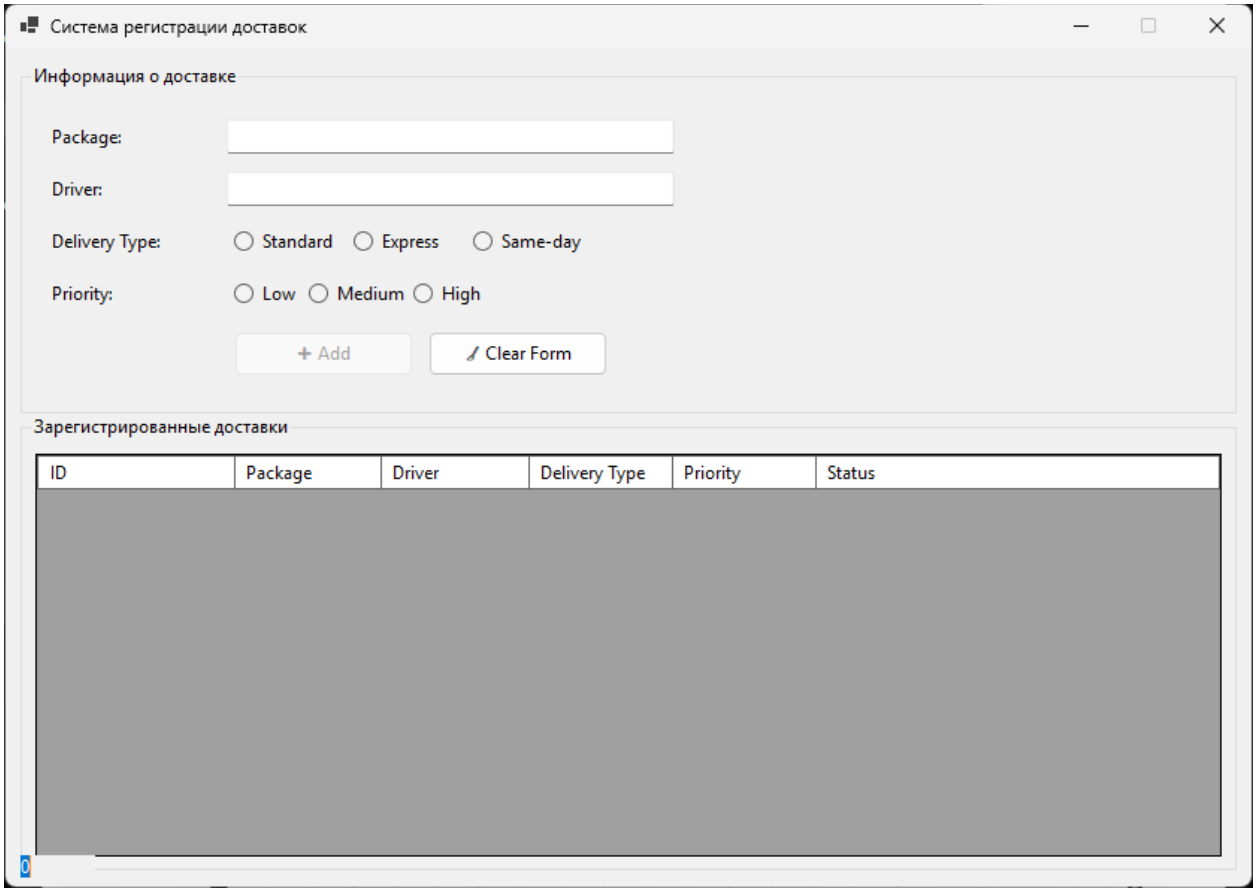
Необходимо реализовать управление доступностью элементов интерфейса:

- кнопка **Add** должна быть **неактивной (IsEnabled = false)**, если не выбрано значение в поле **Delivery Type** (при условии, что все поля обязательны);
- при выборе типа доставки и заполнении всех полей кнопка должна становиться **активной (IsEnabled = true)**;
- кнопка **Clear Form** должна оставаться активной всегда.

📥 Тестовые входные данные (MS Excel)

Для проверки корректности работы приложения и сценария автоматизации необходимо использовать следующий набор данных:

№	Package	Driver	Delivery Type	Priority
1	Document Package	Ivan Petrov	Standard	High
2	Electronics Box	Anna Smirnova	Express	Medium
3	Food Delivery	Dmitry Orlov	Same-day	Low
4	Medical Supplies	Olga Ivanova	Express	High
5	Office Supplies	Sergey Kuznetsov	Standard	Medium



Контрольное задание 2

Задание 2. Базовая автоматизация работы с приложением

После разработки приложения необходимо создать процесс автоматизации в **Puzzle RPA**, имитирующий действия пользователя.

Алгоритм работы робота

1. Запустить разработанное приложение.
2. Дождаться появления главного окна.
3. Последовательно обработать все записи из Excel-таблицы входных данных.

Для каждой записи робот должен:

- ввести значение в поле **Package**;
- ввести значение в поле **Driver**;
- выбрать значение **Delivery Type**;
- выбрать значение **Priority**.

4. Проверить состояние кнопки **Add**:

- если кнопка неактивна (IsEnabled = false), определить незаполненное поле или невыбранный элемент и заполнить его;

- после активации кнопки (IsEnabled = true) нажать её.

5. Повторить действия для всех записей (не менее 5).
6. Проверить появление записей в таблице приложения.

📊 Ожидаемый результат выполнения

После выполнения сценария автоматизации таблица приложения должна содержать записи, соответствующие входным данным.

Пример результата:

ID	Package	Driver	Delivery Type	Priority	Status
1	Document Package	Ivan Petrov	Standard	High	New
2	Electronics Box	Anna Smirnova	Express	Medium	New
3	Food Delivery	Dmitry Orlov	Same-day	Low	New
4	Medical Supplies	Olga Ivanova	Express	High	New
5	Office Supplies	Sergey Kuznetsov	Standard	Medium	New

🔎 Требования к корректности выполнения

Результат выполнения работы считается корректным, если:

✓ Интерфейс приложения

- кнопка **Add** корректно управляется через IsEnabled;
- форма не позволяет добавлять пустые записи.

✓ Работа робота

- робот корректно обрабатывает все входные данные;
- учитывает состояние элементов интерфейса;
- успешно добавляет записи в таблицу.

✓ Данные

- все записи добавлены;
- значения соответствуют входным данным;
- статус каждой записи — "New".

⚠️ Дополнительная проверка устойчивости

Рекомендуется протестировать сценарий с некорректными данными:

Package	Driver	Delivery Type	Priority
(пусто)	Test	Standard	High

Ожидаемое поведение:

- кнопка Add остаётся неактивной;
- робот обнаруживает незаполненное поле;
- корректно завершает добавление записи.

📜 Логирование выполнения процесса

Необходимо реализовать логирование работы робота.

Допускается использование:

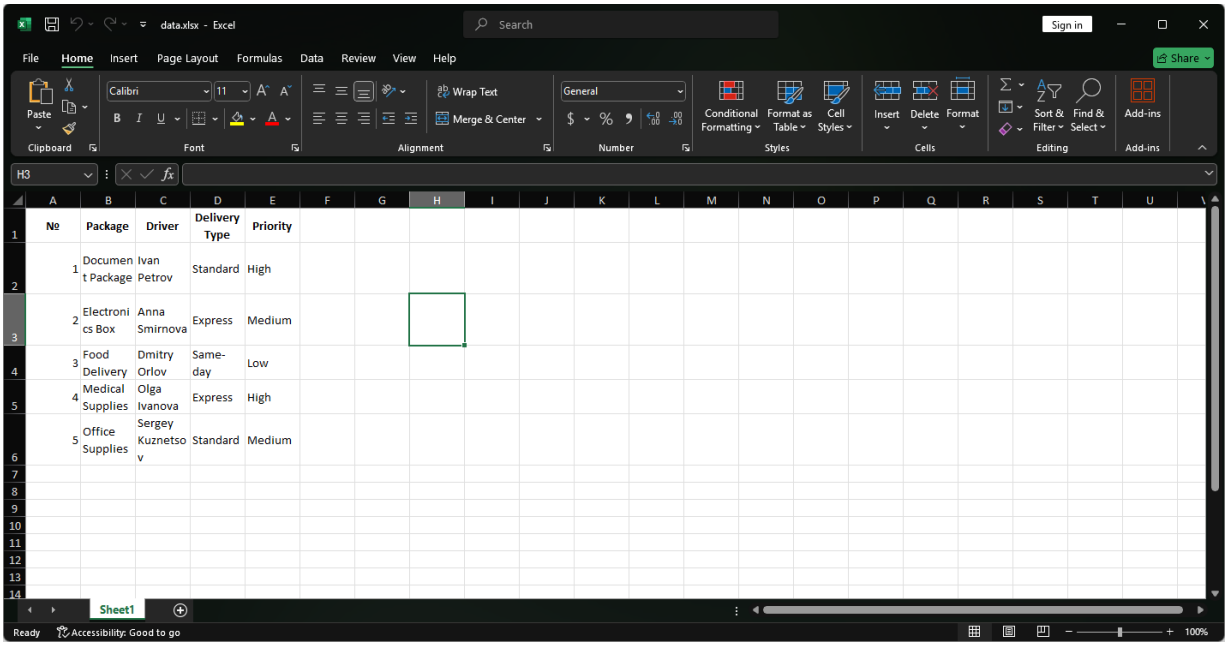
- текстового файла
- файла формата JSON

В журнале должны фиксироваться:

- запуск приложения
- обработка каждой записи
- добавление доставок
- ошибки (при наличии)
- завершение процесса

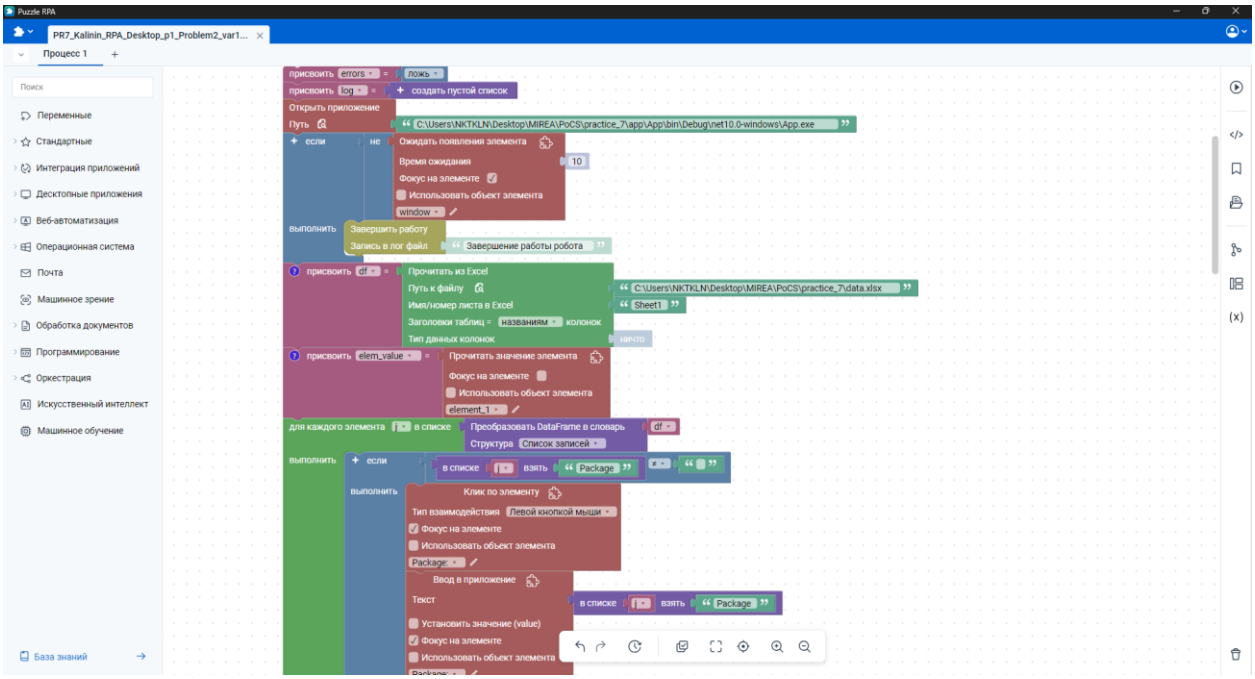
Пример записи:

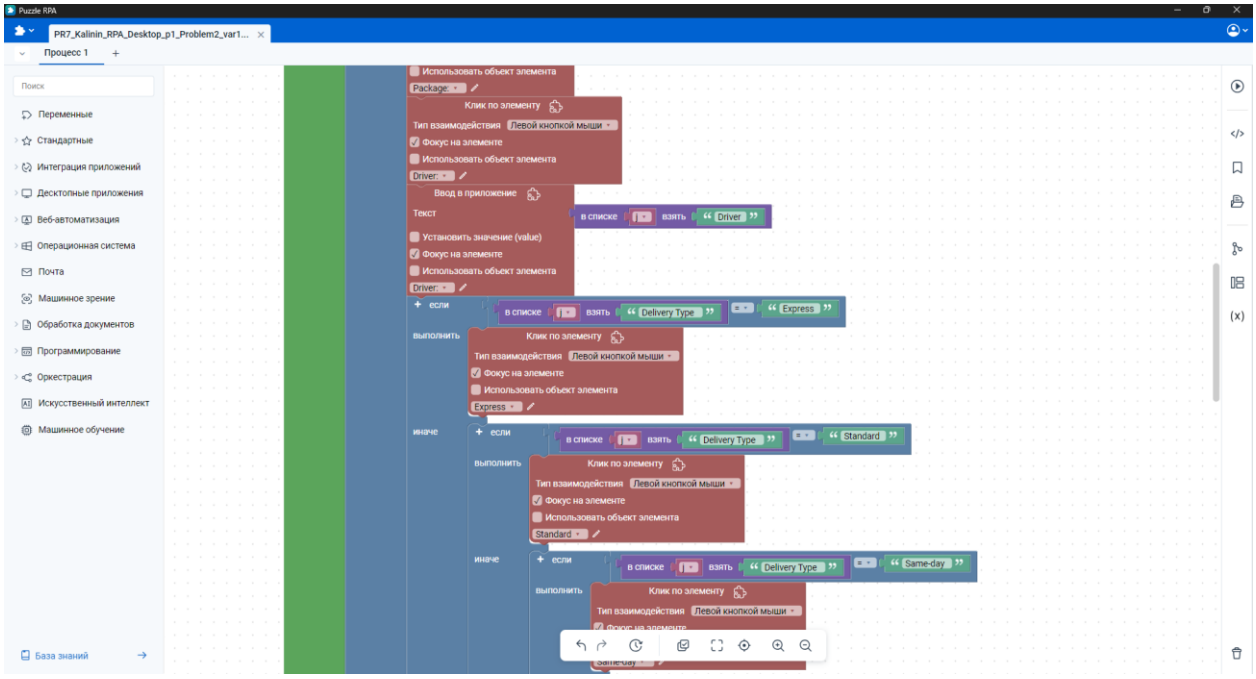
```
{
  "time": "2026-03-15 10:21:04",
  "action": "AddDelivery",
  "package": "Document Package",
  "result": "success"
}
```

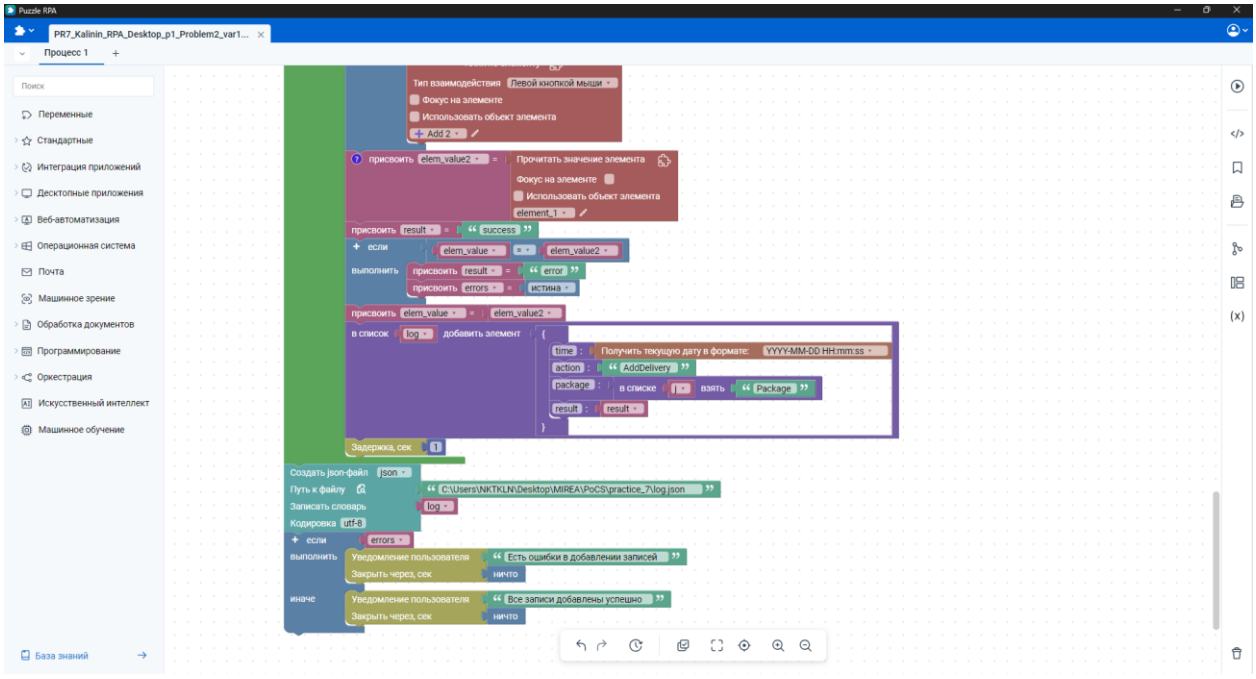


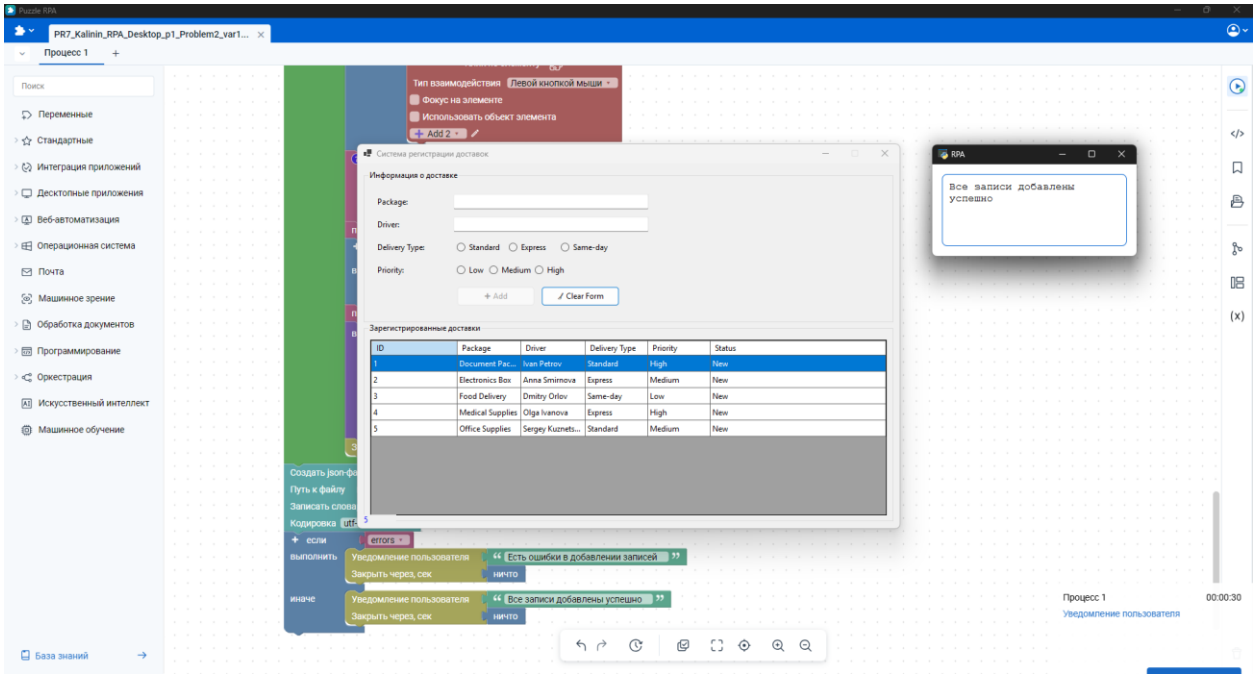
№	Package	Driver	Delivery Type	Priority
1	Document Package	Ivan Petrov	Standard	High
2	Electronics Box	Anna Smirnova	Express	Medium
3	Food Delivery	Dmitry Orlov	Same-day	Low
4	Medical Supplies	Olga Ivanova	Express	High
5	Office Supplies	Sergey Kuznetsov	Standard	Medium

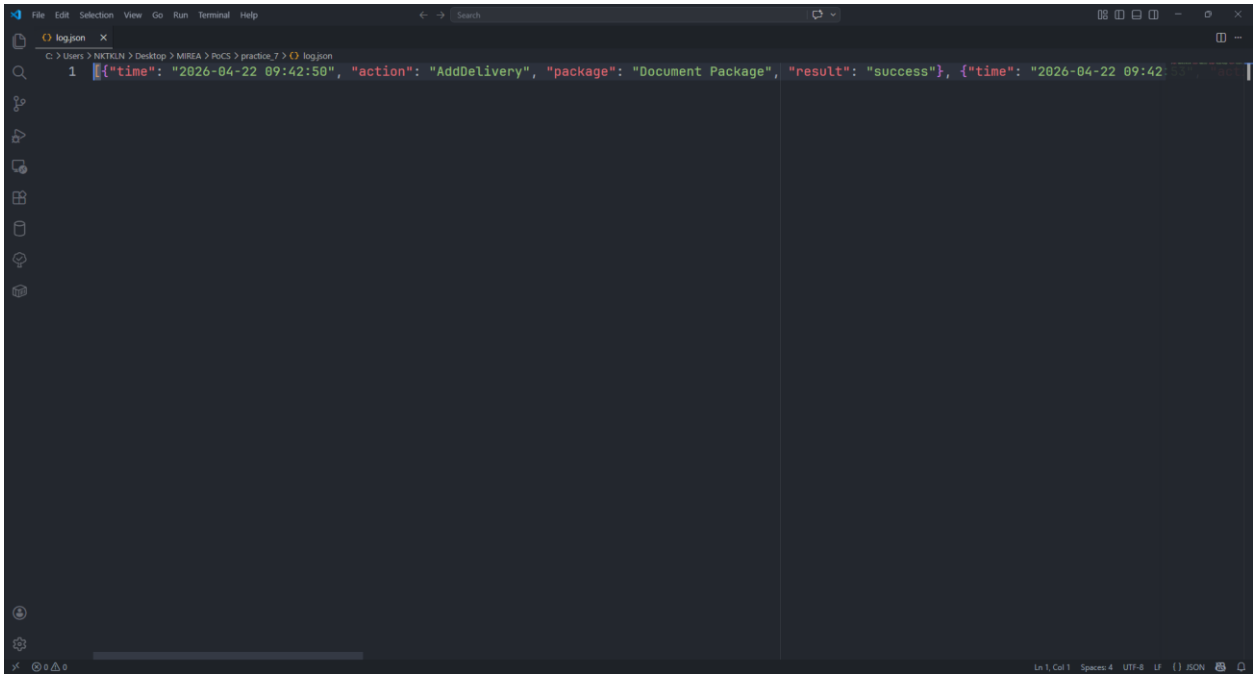
Успешное выполнение кода процесса




Puzzle RPA
PR7_Kalinin_RPA_Desktop_p1_Problem2_var1...
Процесс 1
Поиск
Переменные
Стандартные
Интеграция приложений
Десктопные приложения
Веб-автоматизация
Операционная система
Почта
Машинное зрение
Обработка документов
Программирование
Оркестрация
Искусственный интеллект
Машинное обучение
База знаний
Использовать объект элемента
Package
Клик по элементу
Тип взаимодействия Левой кнопкой мыши
Фокус на элементе
Использовать объект элемента
Driver
Ввод в приложение
Текст
в списке взять Driver
Установить значение (value)
Фокус на элементе
Использовать объект элемента
Driver
если
в списке взять Delivery Type = Express
выполнить
Клик по элементу
Тип взаимодействия Левой кнопкой мыши
Фокус на элементе
Использовать объект элемента
Express
иначе
если
в списке взять Delivery Type = Standard
выполнить
Клик по элементу
Тип взаимодействия Левой кнопкой мыши
Фокус на элементе
Использовать объект элемента
Standard
иначе
если
в списке взять Delivery Type = Same-day
выполнить
Клик по элементу
Тип взаимодействия Левой кнопкой мыши


Puzzle RPA
PR7_Kalinin_RPA_Desktop_p1_Problem2_var1...
Процесс 1
Поиск
Переменные
Стандартные
Интеграция приложений
Десктопные приложения
Веб-автоматизация
Операционная система
Почта
Машинное зрение
Обработка документов
Программирование
Оркестрация
Искусственный интеллект
Машинное обучение
База знаний
Тип взаимодействия Левой кнопкой мыши
Фокус на элементе
Использовать объект элемента
Add 2
присвоить elem_value2 = Прочитать значение элемента
Фокус на элементе
Использовать объект элемента
element_1
присвоить result = success
если elem_value = elem_value2
выполнить присвоить result = error
присвоить errors = истина
присвоить elem_value = elem_value2
в список log добавить элемент
time : Получить текущую дату в формате: YYYY-MM-DD HH:mm:ss
action : AddDelivery
package : в списке взять Package
result : result
Задержка, сек 1
Создать json-файл json
Путь к файлу C:\Users\NKTKLN\Desktop\MIREA\PoCS\practice_7\log.json
Записать словарь log
Кодировка utf-8
если errors
выполнить Уведомление пользователя Есть ошибки в добавлении записей
Закрыть через, сек ничто
иначе Уведомление пользователя Все записи добавлены успешно
Закрыть через, сек ничто


Puzzle RPA
PR7_Kalinin_RPA_Desktop_p1_Problem2_var1...
Процесс 1
Поиск
Переменные
Стандартные
Интеграция приложений
Десктопные приложения
Веб-автоматизация
Операционная система
Почта
Машинное зрение
Обработка документов
Программирование
Оркестрация
Искусственный интеллект
Машинное обучение
База знаний
Тип взаимодействия
Левой кнопкой мыши
Фокус на элементе
Использовать объект элемента
Add 2
Система регистрации доставок
Информация о доставке
Package:
Driver:
Delivery Type:
Standard
Express
Same-day
Priority:
Low
Medium
High
+ Add
Clear Form
Зарегистрированные доставки
ID
Package
Driver
Delivery Type
Priority
Status
1
Document Pac...
Ivan Petrov
Standard
High
New
2
Electronics Box
Anna Smirnova
Express
Medium
New
3
Food Delivery
Dmitry Orlov
Same-day
Low
New
4
Medical Supplies
Olga Ivanova
Express
High
New
5
Office Supplies
Sergey Kuznets...
Standard
Medium
New
RPA
Все записи добавлены успешно
Создать json-фа
Путь к файлу
Записать слова
Кодировка
utf-
если
errors
выполнить
Уведомление пользователя
Есть ошибки в добавлении записей
Закрыть через, сек
ничто
иначе
Уведомление пользователя
Все записи добавлены успешно
Закрыть через, сек
ничто
Процесс 1
00:00:30
Уведомление пользователя


File Edit Selection View Go Run Terminal Help
log.json
C: > Users > NIKTKLN > Desktop > MIREA > PoCS > practice_7 > log.json
1 [{"time": "2026-04-22 09:42:58", "action": "AddDelivery", "package": "Document Package", "result": "success"}, {"time": "2026-04-22 09:42:

Ссылка на репозиторий: https://github.com/NKTKLN/mirea-puzzle-rpa/tree/main/practice_7

Ссылка на чат DeepSeek: https://chat.deepseek.com/share/ulpj2p968jzkrkbqu8